

Appunti di Information Retrieval

Appunti del Corso

Anno Accademico 2026/2027

Indice

0	Introduzione e Architettura dell'Information Retrieval	1
0.1	Definizione e Motivazioni	1
0.2	I Due Macro-Obiettivi: Efficacia ed Efficienza	3
0.3	Architettura di un Sistema di Information Retrieval	3
0.4	Proprietà del Linguaggio e Modelli di Reperimento	4
0.5	L'Inverted Index (Indice Invertito)	5
0.6	Tecniche di Compressione dell'Indice	6
0.7	Learning-to-Rank (LTR) e la Sfida della Non-Differenziabilità	7
0.8	Evoluzione verso il Neural Information Retrieval	7
1	Valutazione dei Sistemi di Information Retrieval	9
1.1	Architettura di un Motore di Ricerca	9
1.2	Come Valutare la Qualità di un Motore di Ricerca	11
1.3	Misurare la Soddisfazione dell'Utente	12
1.4	Il Paradigma di Cranfield e le Collezioni Benchmark	15
1.5	Bisogno Informativo vs. Query	19
1.6	Metriche di Valutazione Non Ordinate (<i>Unranked Measures</i>) .	19
1.7	Metriche Basate sul Ranking (<i>Ranked Measures</i> - Rilevanza Binaria)	22
1.8	Oltre la Rilevanza Binaria: Rilevanza Graduata e NDCG . . .	28
2	Efficienza Algoritmica sulle Architetture CPU Moderne	33
2.1	Il Ruolo dell'Efficienza nell'Information Retrieval	33
2.2	La Gerarchia di Memoria e i Principi di Località	34
2.3	Caso di Studio: Scansione Sequenziale contro <i>Pointer Chasing</i>	37
2.4	Forme di Parallelismo e Architettura della Pipeline	40
2.5	Esecuzione Superscalare, Interleaving e Conflitti Strutturali . .	41
2.6	Data Hazard e Dipendenze Seriali	44
2.7	Control Hazard, Esecuzione Speculativa e Branch Prediction .	45
2.8	Sintesi: dai Vincoli Hardware alle Scelte Algoritmiche	48

3	Elaborazione del Linguaggio Naturale e Rappresentazione del Testo	50
3.1	Recap: Metriche di Valutazione per Rilevanza Graduata	50
3.2	Caratteristiche del Linguaggio Naturale e Legge di Zipf	52
3.3	Acquisizione Dati e Parsing da Web: urllib e BeautifulSoup . .	55
3.4	Pipeline di Pre-elaborazione: Tokenizzazione e Sentence Splitting	57
3.5	Costruzione del Vocabolario e Conteggio	58
3.6	Normalizzazione Morfologica: Stemming vs. Lemmatizzazione	59
3.7	Il Modello Bag of Words (BoW)	61
3.8	N-grammi: Word n-grams e Character n-grams	63
3.9	Il Modello a Spazio Vettoriale (Vector Space Model - VSM) . .	64
4	Modelli di Retrieval e Ranking: Spazio Vettoriale e BM25	66
4.1	Modelli di Retrieval: Booleano vs Ranked	67
4.2	Coefficiente di Jaccard e sue Limitazioni nello Scoring	68
4.3	Matrici Termine–Documento e Ponderazione TF-IDF	69
4.4	Similarità Coseno e Normalizzazione della Lunghezza	72
4.5	Notazione SMART e Varianti di TF-IDF	75
4.6	Best Match 25 (Okapi BM25)	76
4.7	Mappa Concettuale Riassuntiva	78
5	Architettura dei Motori di Ricerca: Indicizzazione e Query Processing	80
5.1	Architettura di Sistema e Paradigma a Due Stadi	80
5.2	Modelli di Scoring: Prodotto Scalare e Connessione con il Dense Retrieval	83
5.3	Strutture Dati ad Alte Prestazioni: Forward Index vs. Indice Invertito	84
5.4	Anatomia, Layout di Memoria e Compressione dell'Indice Invertito	85
5.5	Tipologie di Query, Indici Posizionali e Riscrittura del Testo .	88
5.6	Algoritmi di Query Processing: Term-at-a-Time (TAAT) vs. Document-at-a-Time (DAAT)	89

Capitolo 0

Introduzione e Architettura dell'Information Retrieval

0.1 Definizione e Motivazioni

L'**Information Retrieval** (IR, *Reperimento delle Informazioni*) è la disciplina scientifica e ingegneristica che studia la rappresentazione, la memorizzazione, l'organizzazione e l'accesso a collezioni di dati prevalentemente non strutturati o semi-strutturati, al fine di soddisfare un **bisogno informativo** (*information need*) espresso da un utente [1, 2].

Definizione 0.1.1 (Information Retrieval (Manning et al.)). L'Information Retrieval consiste nel reperire materiale di natura prevalentemente non strutturata (solitamente documenti testuali) che soddisfa un bisogno informativo all'interno di collezioni di grandi dimensioni memorizzate su calcolatori elettronici.

L'IR non si limita ai soli motori di ricerca Web generalisti (Google, Bing), ma costituisce l'infrastruttura abilitante di e-commerce e marketplace, enterprise/desktop search, sistemi **RAG** (*Retrieval-Augmented Generation*, che alimentano i Large Language Model con documenti esterni contestualmente rilevanti) e motori di raccomandazione.

0.1.1 Dai Motori Web all'Agentic Search

L'avvento dei modelli linguistici conversazionali (LLM) non ha reso obsoleto l'Information Retrieval, ma ne ha ampliato l'impatto: un LLM moderno si appoggia costantemente ai principi cardine dell'IR (comprensione semantica della query, matching e ranking, reperimento contestuale) per superare il

limite del *knowledge cutoff*. Nel paradigma **RAG**, il sistema intercetta la richiesta dell’utente, interroga un motore IR, inietta i documenti estratti nel contesto dell’LLM e genera una risposta supportata da citazioni verificabili.

Definizione 0.1.2 (Agentic Search). L’**Agentic Search** è un processo di reperimento dinamico in cui un agente intelligente determina in modo autonomo quali ricerche successive condurre, basandosi sulle evidenze parziali raccolte: ispeziona le fonti, individua lacune conoscitive (*information gaps*), formula nuove query di raffinamento e reitera il ciclo fino alla convergenza informativa.

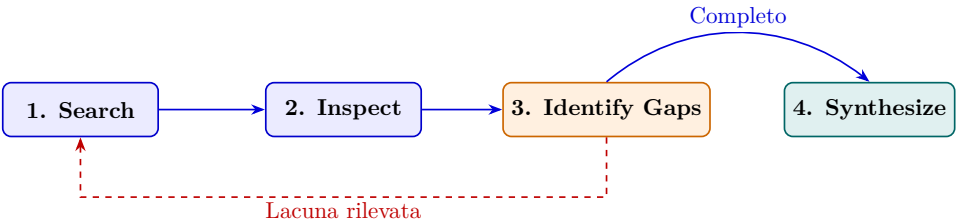


Figura 1: Ciclo a retroazione adattivo (*Adaptive Feedback Loop*) dell’Agentic Search.

0.1.2 Information Retrieval vs. Data Retrieval (Basi di Dati)

Dimensione	Data Retrieval (Database SQL)	Information Retrieval
Struttura dati	Rigidamente schematizzata (tabelle, tipi, vincoli)	Non strutturata o semi-strutturata (testo libero, HTML)
Linguaggio di query	Formale e deterministico (SQL)	Naturale o a parole chiave, spesso ambiguo
Matching	Booleano esatto	Parziale e probabilistico (gradi di rilevanza)
Ordinamento	Nessuno se non esplicito (ORDER BY)	Fondamentale, basato sulla stima di rilevanza
Tolleranza all’errore	Nulla	Elevata (refusi, sinonimia, polisemia)

Tabella 1: Confronto tra Data Retrieval tradizionale e Information Retrieval.

La Rilevanza: Il Cardine dell'IR

Mentre in una base di dati un record soddisfa o viola in modo booleano un predicato logico, nell'Information Retrieval il fulcro è la stima del grado di **rilevanza** rispetto al bisogno informativo dell'utente: soggettiva, situata nel contesto operativo ed evolutiva nel tempo.

0.2 I Due Macro-Obiettivi: Efficacia ed Efficienza

Un sistema di reperimento su larga scala persegue simultaneamente due obiettivi spesso in tensione:

1. **Efficacia (*Effectiveness*)**: quanto accuratamente il ranking soddisfa il bisogno informativo dell'utente.
2. **Efficienza (*Efficiency*)**: costo computazionale, throughput e latenza di risposta, tipicamente nell'ordine dei 10–50 millisecondi per query su archivi di miliardi di documenti.

L'efficienza di un motore di ricerca moderno dipende dall'interazione con l'architettura hardware sottostante: pipelining e processori superscalari, gerarchia di memoria e località di accesso, vettorizzazione SIMD e calcolo parallelo/distribuito su cluster (*sharding*).

0.3 Architettura di un Sistema di Information Retrieval

Un sistema IR adotta una separazione netta tra la **Pipeline Offline** (indicizzazione e addestramento dei modelli di ranking) e la **Pipeline Online** (interrogazione a bassa latenza), come illustrato in Figura 2.

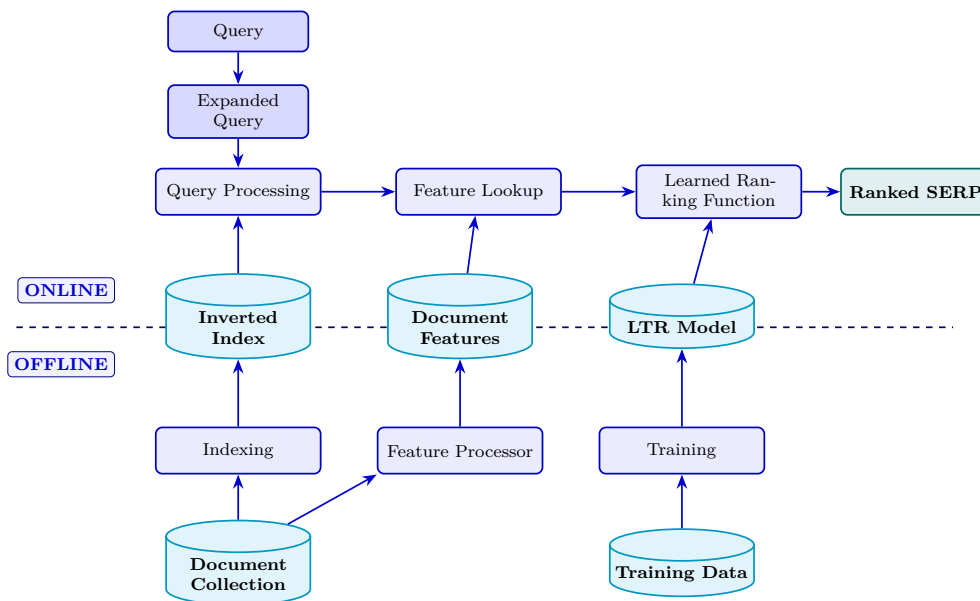


Figura 2: Architettura a griglia di un moderno motore di ricerca (pipeline Online vs. Offline).

0.3.1 Pipeline di Text Processing

Prima dell'indicizzazione, ogni flusso testuale attraversa: **tokenizzazione** (segmentazione in token lessicali), **normalizzazione/case folding** (minuscolo, diacritici), **filtraggio delle stop word** (rimozione di vocaboli ad alta frequenza ma privi di valore discriminante) e **stemming/lemmatizzazione** (riduzione di una forma flessa alla radice o al lemma canonico, es. "searching" → "search").

0.4 Proprietà del Linguaggio e Modelli di Reperimento

- **Legge di Zipf:** la frequenza $f(r)$ di un termine è inversamente proporzionale al suo rango r nella graduatoria di frequenza, $f(r) \propto 1/r$: poche parole frequentissime occupano la maggioranza delle occorrenze, mentre la maggior parte dei termini compare pochissime volte (*long tail*).
- **Legge di Heaps:** la dimensione del vocabolario cresce come legge di potenza rispetto al numero totale di parole N della collezione, $|V| = k \cdot N^\beta$ (con $0.4 \leq \beta \leq 0.6$): il vocabolario non si satura mai completamente.

I principali modelli di reperimento sono il **Modello Booleano** (match esatto tramite operatori logici, privo di punteggio di rilevanza), il **Vector Space Model** (query e documenti come vettori pesati con TF-IDF, rilevanza tramite similarità del coseno) e il **Modello Probabilistico BM25**, standard de facto del reperimento lessicale:

$$\text{score}_{\text{BM25}}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{tf(t, d) \cdot (k_1 + 1)}{tf(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \quad (1)$$

dove $\text{IDF}(t) = \ln\left(\frac{N - df_t + 0.5}{df_t + 0.5} + 1\right)$ pesa la specificità del termine, k_1 (tipicamente 1.2–2.0) controlla la saturazione del term frequency e $b \in [0, 1]$ (tipicamente ≈ 0.75) normalizza la lunghezza del documento $|d|$ rispetto alla lunghezza media avgdl.

0.5 L’Inverted Index (Indice Invertito)

L’**Inverted Index** inverte l’orientamento logico della memorizzazione documentale, realizzando una mappatura $t \mapsto \mathcal{P}_t$, dove $\mathcal{P}_t$ è la lista ordinata dei documenti contenenti il termine t [1].

Definizione 0.5.1 (Inverted Index). Un Inverted Index è costituito da un **dizionario** (l’insieme ordinato dei termini unici, con frequenza documentale df_t e puntatore alla posting list) e, per ogni termine, da una **posting list**: un array ordinato per DocID crescente, i cui elementi possono includere metadati come frequenza locale (tf) e offset posizionali.

Esempio 0.5.1 (Collezione Giocattolo di Riferimento). Si consideri una piccola collezione di tre documenti sull’indicizzazione e la ricerca testuale. La Figura 3 illustra l’Inverted Index generato per alcuni termini chiave: il termine **a** compare nei documenti 1, 2, 3; **an** solo nel documento 1; **index** e **inverted** nei documenti 1, 2; **query** solo nel documento 3.

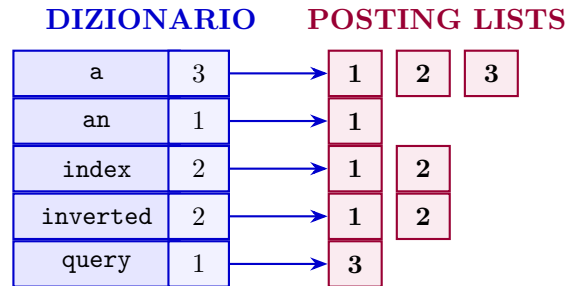


Figura 3: Struttura dell’Inverted Index: dizionario e posting list.

I costrutti booleani (AND, OR, NOT) e la ricerca per frase trovano diretta corrispondenza nell'interfaccia di ricerca avanzata dei motori commerciali (Tabella 2).

Operatore Logico IR	Campo Google Advanced Search	Sintassi
Congiunzione (AND)	<i>all these words</i>	tri-colour rat terrier
Ricerca per frase	<i>this exact word or phrase</i>	"rat terrier"
Disgiunzione (OR)	<i>any of these words</i>	miniature OR standard
Negazione (NOT)	<i>none of these words</i>	-rodent -"Jack Russell"
Intervallo numerico	<i>numbers ranging from...to</i>	10..35 kg

Tabella 2: Mappatura tra operatori booleani IR e Ricerca Avanzata di Google.

0.6 Tecniche di Compressione dell'Indice

La compressione dell'Inverted Index riduce drasticamente l'occupazione di memoria (sia su RAM che su disco), convertendo un collo di bottiglia di I/O in un carico di decompressione CPU, generalmente molto più veloce.

0.6.1 Binary Interpolative Coding

Il **Binary Interpolative Coding** (Moffat e Stuiver [3]) comprime sequenze strettamente crescenti di interi (i DocID delle posting list) partizionandole ricorsivamente: data una sottolista di n interi nota per essere compresa in $[l, h]$, si codifica l'elemento centrale x_m (con $m = \lfloor (n+1)/2 \rfloor$) usando $\lceil \log_2 R \rceil$ bit, dove $R = h - l - n + 1$ è il numero di valori ammissibili, e si ricorre poi separatamente sulle due metà. Quando i DocID sono densamente clusterizzati, R tende a 1 e molti elementi vengono codificati a costo pressoché nullo.

0.6.2 Compressione Basata su Dizionario: LZ77

Per testi generali, l'algoritmo **Lempel-Ziv '77** (LZ77) comprime tramite una finestra scorrevole, sostituendo ripetizioni lessicali con triple costituite da (offset, lunghezza, carattere) riferite a occorrenze precedenti nella finestra. Questa famiglia di algoritmi è alla base di strumenti diffusi come **gzip**, **7-Zip** e **LZ4**.

0.7 Learning-to-Rank (LTR) e la Sfida della Non-Differenziabilità

I candidati selezionati dall’Inverted Index vengono inoltrati a un modello di Machine Learning che li riordina (**Learning-to-Rank**, LTR) [4]. Il dataset di addestramento associa a ciascuna query una lista di documenti con giudizio di rilevanza graduato $y \in \{0, \dots, 4\}$ (da irrilevante a perfettamente rilevante); un modello calcola uno score $s_i = f_\theta(\vec{x}_{q,d_i})$ per ciascuna coppia query-documento, e la lista finale viene ottenuta ordinando per punteggio decrescente.

Il Cardine dell’LTR

The goal is to learn the ranking, not the label: l’obiettivo non è stimare accuratamente il singolo valore y , ma l’ordinamento relativo ottimale, massimizzando metriche cumulative come **NDCG** o **MAP**.

Poiché l’operazione di ordinamento è discreta e a gradini, il gradiente delle metriche di ranking rispetto ai parametri del modello è nullo quasi ovunque: non è possibile applicare direttamente la discesa del gradiente. La letteratura ha sviluppato tre famiglie di soluzioni:

1. **Pointwise:** tratta ogni documento come un’istanza indipendente (regressione o classificazione su y), ignorando la natura comparativa del ranking.
2. **Pairwise** (es. RankNet): confronta coppie di documenti, modellando con una sigmoide differenziabile la probabilità che d_i preceda d_j .
3. **Listwise** (es. LambdaMART): ottimizza l’intera lista tramite gradienti surrogati λ_{ij} , pesati con la variazione effettiva di NDCG derivante dallo scambio di d_i e d_j ; combinato con alberi di decisione potenziati, è stato per oltre un decennio lo standard nei motori di ricerca commerciali.

0.8 Evoluzione verso il Neural Information Retrieval

Dal 2018, i modelli Transformer (BERT) hanno inaugurato l’era del Neural Information Retrieval [5, 6], pre-addestrati tramite **Masked Language Modeling**. Gli approcci neurali per il reperimento si dividono in due famiglie:

- **Interazione (Cross-Encoder):** query e documento sono concatenati ed elaborati congiuntamente da un Transformer, catturando interazioni

profonde a costo computazionale elevato ($\mathcal{O}(|\mathcal{D}| \times \text{Transformer})$); usati solo come re-ranker su un piccolo insieme di candidati.

- **Rappresentazione (Bi-Encoder):** query e documento sono proiettati indipendentemente in uno spazio vettoriale da due encoder distinti; i vettori dei documenti sono pre-calcolabili offline, riducendo il matching online a un prodotto scalare o a una ricerca di vicini approssimati.

Tra i modelli a rappresentazione si distinguono tre famiglie di embedding: **dense single-vector** (un unico vettore denso per documento/query, interrogato con indici k-ANN su grafi come HNSW), **dense multi-vector** (es. ColBERT [5], un vettore per token con aggregazione *Late Interaction*/MaxSim, $S(q, d) = \sum_{t_q \in q} \max_{t_d \in d} (\vec{E}_{t_q} \cdot \vec{E}_{t_d}^T)$) e **sparse single-vector** (es. SPLADE [6], interpretabile per design e più efficiente in spazio/tempo rispetto al multi-vector).

k-Approximate Nearest Neighbor Search

Su collezioni di milioni di documenti rappresentati come vettori densi ad alta dimensionalità, il recupero esatto dei k più simili alla query è computazionalmente proibitivo: si ricorre quindi a indici di **ricerca approssimata dei vicini** (k -ANN), che scambiano una minima perdita di accuratezza per un guadagno di ordini di grandezza in velocità. **TusKANNy** (<https://tuskanny.com/>), sviluppato da ISTI-CNR e Università di Pisa, ne è un esempio open-source ad alte prestazioni scritto in Rust.

Capitolo 1

Valutazione dei Sistemi di Information Retrieval

1.1 Architettura di un Motore di Ricerca

Prima di affrontare la valutazione, è utile inquadrare l'architettura complessiva di un moderno motore di ricerca, che può essere schematizzata nel seguente diagramma a blocchi:

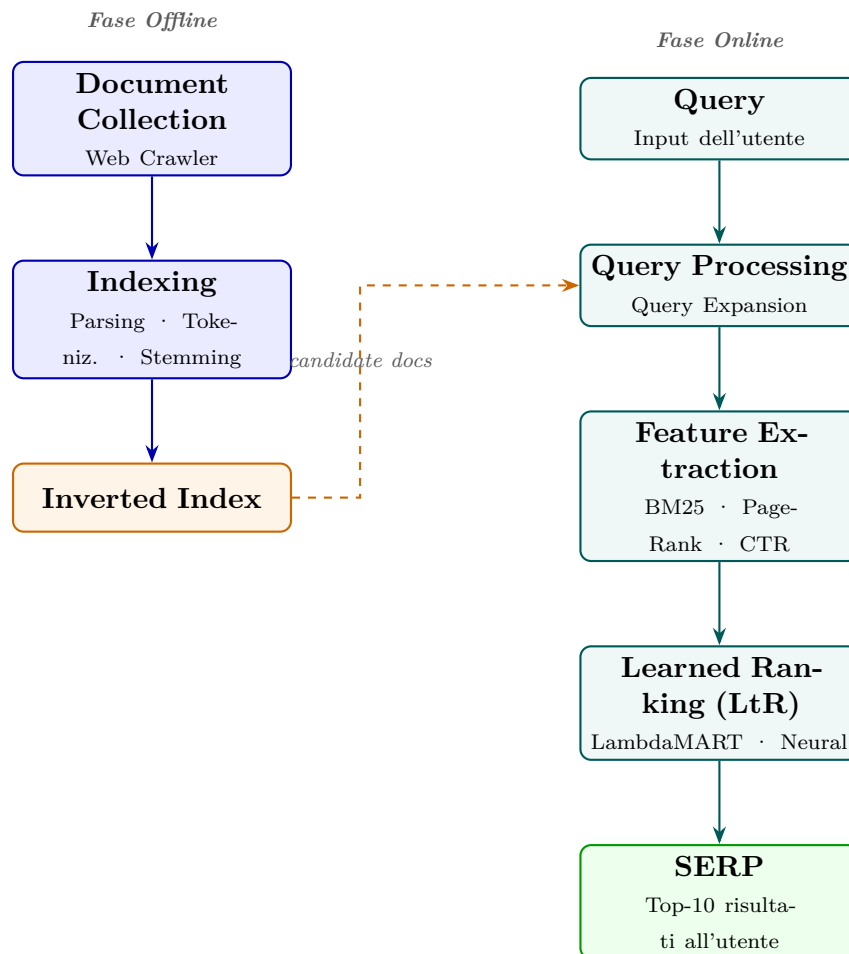


Figura 1.1: Diagramma a blocchi dell'architettura di un motore di ricerca moderno: pipeline offline (indicizzazione) e pipeline online (elaborazione query e ranking).

I componenti principali sono:

- **Web Crawler (Document Collection):** Un agente automatico che percorre il Web seguendo i collegamenti ipertestuali e scarica i documenti da indicizzare. La collezione risultante è la base di lavoro del sistema.
- **Indexing:** La pipeline di pre-elaborazione che trasforma i documenti grezzi in una rappresentazione indicizzabile: parsing HTML, tokenizzazione, normalizzazione, rimozione delle stopword e stemming/lemmatizzazione.

- **Inverted Index:** La struttura dati centrale del motore di ricerca: per ogni termine mappa la lista dei documenti che lo contengono (posting list), consentendo un recupero efficiente.
- **Query Processing e Query Expansion:** Il modulo che analizza la query dell'utente, la normalizza e — facoltativamente — la espande con sinonimi o termini correlati per aumentare il richiamo.
- **Feature Extraction:** Estrazione di segnali numerici per ciascuna coppia query-documento (TF-IDF, BM25, PageRank, freschezza, segnali di click, ecc.).
- **Learned Ranking Function (Learning to Rank):** Un modello statistico o di apprendimento automatico (ad es. LambdaMART, LambdaRank) addestrato a ordinare i documenti massimizzando una metrica di qualità, tipicamente NDCG.
- **SERP (Search Engine Results Page):** La pagina dei risultati presentata all'utente, comprensiva di titolo, snippet, URL e, nei motori moderni, risposte generate (Featured Snippets, AI Overviews).

1.2 Come Valutare la Qualità di un Motore di Ricerca

Un sistema di Information Retrieval è un artefatto complesso che integra decine di componenti eterogenei: come si valuta qualcosa di così articolato? La risposta della comunità scientifica distingue nettamente due dimensioni complementari:

1. Metriche di Efficienza:

- *Velocità di indicizzazione:* Quanti documenti per ora il sistema è in grado di processare e inserire nell'indice.
- *Indicizzazione incrementale:* Capacità di aggiornare l'indice con nuovi flussi documentali (ad es. +10 000 prodotti al giorno in un e-commerce) senza interruzioni di servizio.
- *Velocità di ricerca (Latenza e Throughput):* Tempo di risposta (latenza al 95° o 99° percentile) e consumo di CPU/memoria nell'elaborazione di migliaia di query al secondo su collezioni di svariati milioni o miliardi di documenti.

2. Metriche di Efficacia e Qualità dei Servizi Accessori:

- Qualità delle raccomandazioni correlate, accuratezza della correzione ortografica delle query (*spell checking*) e chiarezza dei riassunti testuali visualizzati (*snippet generation*).
- **Efficacia intrinseca della graduatoria:** Misura la capacità fondamentale del motore di ricerca di soddisfare l'utente, ordinando i documenti pertinenti nelle prime posizioni della pagina dei risultati (*Search Engine Results Page*, SERP).

1.3 Misurare la Soddisfazione dell'Utente

Nel contesto operativo dei motori di ricerca commerciali, l'obiettivo ultimo è massimizzare la soddisfazione dell'utente (*User Happiness*). Poiché la felicità è uno stato cognitivo non osservabile direttamente, i sistemi di produzione ricorrono a proxy comportamentali online:

- **Clic e Click-Through Rate (CTR):** Il numero e la proporzione di clic ricevuti dai risultati restituiti.
 - *Criticità:* Titoli sensazionalistici o riassunti ingannevoli (*clickbait*) inducono clic elevati senza che il documento soddisfi realmente l'utente.
 - *La ricerca a zero clic (Zero-Click Search):* L'assenza di clic non è necessariamente un segnale negativo. Quando un utente cerca informazioni fattuali (ad es. il meteo, l'orario di un volo, una conversione di valuta o una definizione enciclopedica), la risposta visualizzata direttamente tramite *Featured Snippet* o *Knowledge Graph* soddisfa istantaneamente il bisogno senza richiedere alcun clic.
- **Conversioni e Transazioni Economiche:** Negli e-commerce, la quota di sessioni di ricerca che sfociano nell'acquisto di un prodotto e l'ammontare monetario speso dall'utente costituiscono metriche primarie di business.
- **Fidelizzazione e Visite Ripetute:** La percentuale di utenti che tornano ad adoperare il motore di ricerca entro un intervallo temporale predefinito (una settimana, un mese).

- **Dwell Time (Tempo di Permanenza):** L'intervallo di tempo che intercorre tra il momento in cui l'utente clicca su un risultato della SERP e l'istante in cui fa ritorno alla pagina dei risultati (*bouncing back*). Un Dwell Time prolungato indica generalmente che il documento ha catturato l'attenzione ed è risultato informativo, mentre un ritorno immediato alla SERP denota insoddisfazione.

La Valutazione nell'Era dei Motori Generativi e LLM

L'emergere di motori di ricerca generativi e assistenti basati su Large Language Models (LLM) ridefinisce radicalmente metriche consolidate come Dwell Time e CTR: l'utente non naviga più necessariamente attraverso una lista di link esterni, ma consuma sintesi testuali generate direttamente dal modello. In tale scenario, l'accuratezza fattuale, l'assenza di allucinazioni e la corretta attribuzione delle fonti rimpiazzano il conteggio dei clic tradizionali.

Il dibattito metodologico su come valutare questi sistemi solleva interrogativi cruciali:

- **Numero di turni conversazionali come proxy di insoddisfazione:** In un sistema conversazionale, un elevato numero di turni necessari per soddisfare un bisogno informativo può segnalare che il sistema non ha compreso l'intento iniziale dell'utente, costringendolo a continue riformulazioni. Il Dwell Time perde quindi centralità, sostituito dall'analisi dell'efficienza e della lunghezza della sessione di dialogo.
- **Varianza e instabilità del prompting:** A differenza delle query lessicali dell'IR classico, lievi variazioni nella formulazione del prompt possono indurre forti discrepanze qualitative nella risposta dell'LLM. Questa elevata sensibilità rende complessa la standardizzazione e la riproducibilità sperimentale dei benchmark.
- **Differenza ontologica tra recupero e sintesi:** Un motore tradizionale si limita a *recuperare* e ordinare documenti preesistenti da un corpus noto; un modello generativo *sintetizza* invece un testo originale probabilistico. Mentre nel primo caso la rilevanza è misurabile rispetto a un insieme oggettivo di documenti rilevanti (*ground truth*), nel secondo la correttezza fattuale della sintesi richiede verifiche semantiche inedite.
- **Il paradigma RAG (Retrieval-Augmented Generation):** L'architettura RAG vincola la generazione alla consultazione preventiva di documenti estratti da un indice IR tradizionale. In questo contesto, la valutazione diventa composita: occorre misurare separatamente e congiuntamente l'efficacia del recupero documentale e la fedeltà informativa del testo generato.

1.4 Il Paradigma di Cranfield e le Collezioni Benchmark

Sebbene i segnali online forniscano un feedback prezioso, essi risultano rumorosi, soggetti a forti bias di posizionamento (*Position Bias*) e non riproducibili sperimentalmente prima del rilascio in produzione. Per consentire valutazioni comparative controllate, la ricerca in IR adotta il **Paradigma di Cranfield**, sviluppato originariamente da Cyril Cleverdon negli anni '60 [7].

È importante contestualizzare il momento storico: Cleverdon lavorava su **biblioteche fisiche**, con schedari cartacei e cataloghi aeronautici. Il suo esperimento — condotto prima dell'avvento dei calcolatori elettronici diffusi — consisteva nel valutare manualmente quanto bene diversi sistemi di classificazione documentale riuscissero a soddisfare bisogni informativi reali. Eppure, il framework metodologico che ne derivò è rimasto il fondamento della valutazione in IR per oltre sessant'anni.

Definizione 1.4.1 (Collezione di Test (Test Collection)). Una collezione di test standard per l'Information Retrieval è una tripletta formale $\mathcal{T} = (\mathcal{D}, \mathcal{Q}, \mathcal{J})$ costituita da:

1. Una collezione documentale benchmark $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$;
2. Una suite benchmark di bisogni informativi e query $\mathcal{Q} = \{q_1, q_2, \dots, q_M\}$;
3. Un insieme di giudizi di rilevanza (*Relevance Judgments* o *grels*) $\mathcal{J} = \{(q, d) \mapsto r_{q,d}\}$, che associa a ciascuna coppia query-documento un valore di rilevanza (binario o graduato).

1.4.1 La Sfida della Scalabilità Combinatoria

La realizzazione manuale esaustiva dei giudizi di rilevanza è materialmente impossibile per archivi realistici. Si consideri uno scenario benchmark con:

$$|\mathcal{D}| = 5 \times 10^6 \text{ documenti}, \quad |\mathcal{Q}| = 50\,000 \text{ query}$$

Lo spazio complessivo delle coppie ammonta a:

$$|\mathcal{D}| \times |\mathcal{Q}| = 5 \times 10^6 \times 5 \times 10^4 = 2.5 \times 10^{11} \text{ coppie query-documento}$$

Ipotizzando che un annotatore umano impieghi in media 2.5 secondi per valutare la rilevanza di un documento:

$$\begin{aligned} \text{Tempo Totale} &= 2.5 \times 10^{11} \times 2.5 \text{ s} = 6.25 \times 10^{11} \text{ secondi} \\ &\approx 173\,611\,111 \text{ ore/persona} \end{aligned}$$

Un impegno titanico che richiederebbe decine di migliaia di anni di lavoro umano ininterrotto! Per ovviare a ciò, i benchmark limitano i giudizi a un sottoinsieme accuratamente campionato di documenti.

1.4.2 Origini delle Query di Test

Una questione metodologica fondamentale riguarda la provenienza delle query che compongono il benchmark $\mathcal{Q}$:

- **Motori Web commerciali — Query Logs:** I grandi motori di ricerca (Google, Microsoft Bing) campionano le query di test direttamente dai propri *query log*, cioè dai log di ricerca reali degli utenti. Dataset pubblici storici come l'AOL Query Log (2006) e i dataset Microsoft ORCAS/MIMICS contengono milioni di query reali. Questo approccio garantisce che le query riflettano bisogni informativi autentici con la distribuzione di frequenza reale (incluse le query rare della coda lunga).
- **Scenario classico — Esperti a tavolino:** Nelle prime collezioni (Cranfield, TREC), esperti di dominio stilavano manualmente liste di *information needs* plausibili, simulando i bisogni di un ipotetico utente. Il processo era deliberato e supervisionato, con attenzione a coprire diversi tipi di bisogno informativo (noto, esplorativo, transazionale).
- **Perché non generare query casuali dai documenti:** Una strategia apparentemente ovvia — campionare termini o frasi dai documenti della collezione per costruire query sintetiche — è metodologicamente scorretta. Le query così generate non rifletterebero gli intenti reali degli utenti, ma solo il vocabolario dei documenti, producendo una valutazione distorta e priva di validità ecologica.

1.4.3 Crowdsourcing: Opportunità e Limiti

Una strategia economica consiste nel delegare la valutazione a lavoratori retribuiti su piattaforme online di crowdsourcing (come Amazon Mechanical Turk). Tuttavia, la letteratura scientifica evidenzia come tali giudizi siano affetti da:

- Elevatissima varianza statistica;
- Minore competenza e presenza di spammer;
- Necessità di filtri rigorosi di consenso (ad es. votazione di maggioranza o calcolo dell'accordo inter-annotatore tramite coefficiente κ di Cohen).

Una strategia più recente e sempre più diffusa è l'impiego di **LLM come Valutatori** (*LLM-as-a-Judge*):

- Modelli come GPT-4 vengono utilizzati con prompt strutturati per valutare la rilevanza di coppie query-documento, in sostituzione o affiancamento agli annotatori umani. La scalabilità economica è enormemente superiore al crowdsourcing tradizionale: il costo per giudizio scende di uno o due ordini di grandezza.
- Tuttavia, questa metodologia presenta **bias intrinseci** documentati: preferenza per testi più lunghi (indipendentemente dalla qualità), *self-preference bias* (i modelli tendono a valutare meglio testi simili ai propri output di addestramento) e sensibilità all'ordine di presentazione dei candidati. È quindi necessario progettare con cura i prompt di valutazione e condurre studi di calibrazione rispetto ai giudizi umani.

1.4.4 L'Infrastruttura TREC (*Text REtrieval Conference*)

Promossa dal National Institute of Standards and Technology (NIST), la conferenza **TREC** rappresenta il punto di riferimento internazionale per la valutazione delle tecnologie di recupero dell'informazione. TREC è strutturata in tracce tematiche (*Tracks*), ciascuna incentrata su specifici casi d'uso (ricerca Web, ricerca clinica, domande-risposte, deep retrieval). Il risultato prodotto da un sistema software che esegue un compito su una collezione di test prende il nome di **Run**.

La Tabella 1.1 documenta l'evoluzione dimensionale delle collezioni benchmark storiche utilizzate nella ricerca in Information Retrieval [1], evidenziando il salto di scala che ha reso indispensabile la nascita di TREC.

Collezione	Documenti (<i>N</i>)	Query	Dimensione (MB)	Termini/Doc	Giudizi (Q-D)
ADI	82	35	—	—	—
AIT	2 109	14	2	400.0	> 10 000
CACM	3 204	64	2	24.5	—
CISI	1 460	112	2	46.5	—
Cranfield	1 400	225	2	53.1	Completi
LISA	5 872	35	3	—	—
Medline	1 033	30	1	—	—
NPL	11 429	93	3	—	—
OSHMED	348 566	106	400	250.0	16 140
Reuters	21 578	672	28	131.0	—
TREC (Tipica)	740 000	200	2 000 (2 GB)	89–3 543	> 100 000

Tabella 1.1: Panoramica comparativa dei principali corpora benchmark storici di valutazione.

Come si evince dai dati, nelle prime collezioni degli anni '60 e '70 (come Cranfield e Medline), le ridotte dimensioni (poche migliaia di record e 1–2 MB) consentivano un'annotazione esaustiva manuale. Con l'avvento di collezioni realistiche da centinaia di migliaia di documenti e gigabyte di testo come TREC, lo spazio combinatorio delle valutazioni è esploso, imponendo l'adozione di metodologie di campionamento selettivo.

1.4.5 La Tecnica del Pooling

Per campionare i documenti da sottoporre ad annotazione umana senza dover valutare l'intera collezione, nei benchmark TREC si adotta la tecnica del **Pooling** (Figura 1.2):

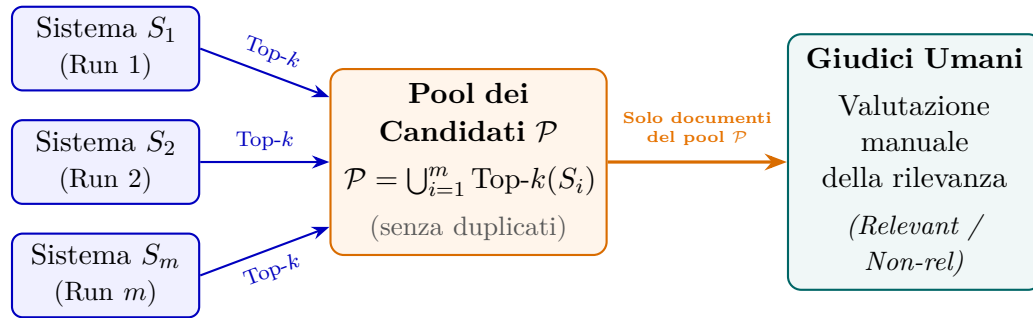


Figura 1.2: Processo di Pooling per la selezione controllata dei documenti da annotare.

1. Si raccolgono le risposte fornite da m sistemi di recupero eterogenei per una determinata query.
2. Si estraggono i primi k documenti restituiti da ciascun sistema (tipicamente $k = 50$ o $k = 100$).
3. Si calcola l'**unione** di tali insiemi eliminando i duplicati, formando il pool $\mathcal{P}$.
4. Gli annotatori umani valutano la rilevanza unicamente per i documenti appartenenti a $\mathcal{P}$.

L'Assunzione Fondamentale del Pooling

Tutti i documenti dell'archivio che non compaiono nel pool (*unpooled*) vengono assunti convenzionalmente come **non rilevanti** ($r = 0$). Sebbene tale assunzione introduca un potenziale bias a sfavore di algoritmi futuri con paradigmi radicalmente differenti, studi empirici condotti da NIST confermano che, a fronte di un pool formato da sistemi sufficientemente diversificati, le conclusioni comparative tra motori di ricerca rimangono stabili e affidabili.

1.5 Bisogno Informativo vs. Query

Una distinzione concettuale decisiva riguarda il divario tra bisogno informativo e formulazione della query:

- **Bisogno Informativo (*Information Need*)**: Rappresenta lo stato cognitivo e l'obiettivo reale dell'utente. Ad esempio: *"Il fondo della mia piscina sta diventando nero e necessita di essere ripulito"*.
- **Query**: L'espressione testuale sintetica digitata nel box di ricerca. Ad esempio: "pulitore piscina".

Proprietà 1.5.1 (Principio di Valutazione della Rilevanza). *La rilevanza di un documento deve essere sempre stimata rispetto all'**Information Need** sottostante, mai rispetto alla mera sequenza di caratteri che compone la query.*

1.6 Metriche di Valutazione Non Ordinate (*Unranked Measures*)

Nel contesto del reperimento non ordinato (come nel modello booleano classico), il sistema restituisce un insieme non graduato di documenti. La valutazione si fonda sulla suddivisione dell'universo documentale mediante la matrice di contingenza (Tabella 1.2):

	Documento Rilevante	Documento Non Rilevante
Documento Reperito	True Positive (TP)	False Positive (FP)
Documento Non Reperito	False Negative (FN)	True Negative (TN)

Tabella 1.2: Matrice di contingenza per la classificazione binaria query-documento.

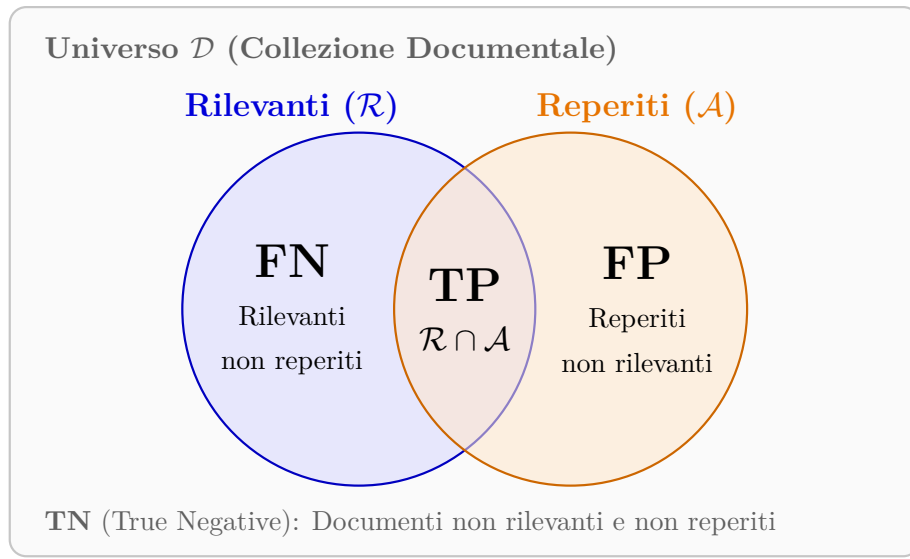


Figura 1.3: Rappresentazione insiemistica del reperimento documentale e delle regioni della matrice di confusione.

1.6.1 Precisione, Richiamo e loro Trade-off

Definizione 1.6.1 (Precisione ($Precision$, P)). La frazione dei documenti restituiti dal sistema che risultano effettivamente rilevanti:

$$P = \frac{TP}{TP + FP} = P(\text{rilevante} \mid \text{reperito}) \quad (1.1)$$

Definizione 1.6.2 (Richiamo ($Recall$, R)). La frazione dei documenti rilevanti complessivamente presenti nella collezione che il sistema è riuscito a reperire:

$$R = \frac{TP}{TP + FN} = P(\text{reperito} \mid \text{rilevante}) \quad (1.2)$$

Esiste un intrinseco **trade-off** tra Precisione e Richiamo:

- Un sistema che restituisce l'intera collezione documentale (Retrieved = $\mathcal{D}$) ottiene banalmente un richiamo perfetto $R = 100\%$, ma la sua precisione crolla a livelli minimi ($P \approx 0$).
- Viceversa, un sistema conservativo che restituisce unicamente il singolo documento più sicuro può conseguire $P = 100\%$, pagando il prezzo di un richiamo trascurabile.

1.6.2 La Media Armonica: F-Measure (F_1 -Score)

Per sintetizzare Precisione e Richiamo in un unico valore scalare che premi l'equilibrio tra le due grandezze, si adotta la **F-Measure** (o F_1 -score), definita come la **media armonica** di P ed R :

$$F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2 \cdot P \cdot R}{P + R} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \quad (1.3)$$

Proprietà 1.6.1 (Proprietà della Media Armonica). *La media armonica gode di proprietà matematiche fondamentali per la valutazione:*

1. È sempre minore o uguale alla media geometrica e alla media aritmetica:

$$\frac{2}{\frac{1}{a} + \frac{1}{b}} \leq \sqrt{a \cdot b} \leq \frac{a + b}{2}$$

2. Quando i due argomenti differiscono notevolmente, la media armonica collassa verso il **valore minimo** anziché posizionarsi a metà strada.

Esempio 1.6.1 (Comportamento Limite dell'F-Measure). Si consideri un sistema banale che restituisce l'intera collezione: $R = 1.0$ e $P = 0.0001$.

- La media aritmetica assegna un punteggio fuorviante pari a:

$$\frac{1.0 + 0.0001}{2} \approx 0.50 \quad (50\%)$$

- La media armonica (F_1) demolisce giustamente la valutazione:

$$F_1 = \frac{2 \cdot 1.0 \cdot 0.0001}{1.0 + 0.0001} \approx 0.0002 \quad (0.02\%)$$

Ciò dimostra per quale motivo la media aritmetica sia inadeguata e l'F-measure costituisca la scelta corretta.

1.6.3 La Misura F_β Pesata

Qualora si desideri attribuire pesi differenziati a Precisione e Richiamo, si introduce un parametro di ponderazione $\alpha \in [0, 1]$:

$$F = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}} \quad (1.4)$$

Esprimendo α in funzione di un parametro β tale che $\alpha = \frac{1}{1+\beta^2}$, si ricava la formulazione classica di F_β :

$$F_\beta = \frac{(1 + \beta^2) \cdot P \cdot R}{\beta^2 \cdot P + R} \quad (1.5)$$

- Valori di $\beta < 1$ (ad es. $\beta = 0.5$) enfatizzano la **Precisione** rispetto al Richiamo (utile nelle ricerche su Web generico).
- Valori di $\beta > 1$ (ad es. $\beta = 2.0$) enfatizzano il **Richiamo** rispetto alla Precisione (fondamentale nelle ricerche brevettuali, legali e mediche, dove omettere un documento rilevante comporta danni gravi).

1.7 Metriche Basate sul Ranking (*Ranked Measures* - Rilevanza Binaria)

Nei motori di ricerca moderni i documenti non vengono restituiti come insiemi informi, ma ordinati in base a uno score di rilevanza decrescente. Inoltre, gli utenti Web raramente oltrepassano la prima pagina dei risultati (i primi 10 o 20 elementi).

1.7.1 Precision@k ($P@k$) e Recall@k ($R@k$)

Definizione 1.7.1 (Precision@k). Fissata una soglia di rango $k \in \mathbb{N}^+$, la Precision@k corrisponde alla proporzione di documenti rilevanti presenti nei primi k posti della graduatoria:

$$P@k = \frac{1}{k} \sum_{i=1}^k \text{rel}_i \quad (1.6)$$

dove $\text{rel}_i \in \{0, 1\}$ denota la rilevanza binaria del documento al rango i .

Esempio 1.7.1 (Calcolo di Precision@k). Si consideri una lista ordinata in cui la sequenza di rilevanza binaria dei primi 5 documenti è data da $\langle 1, 1, 0, 0, 1 \rangle$:

- $P@1 = \frac{1}{1} = 1.00$
- $P@2 = \frac{2}{2} = 1.00$
- $P@3 = \frac{2}{3} \approx 0.66$
- $P@4 = \frac{2}{4} = 0.50$ (si noti che $P@4 < P@3$)
- $P@5 = \frac{2}{5} = 0.40$

La Distorsione di $P@k$ nella Media tra Query

Sebbene $P@k$ sia intuitivo per stimare la qualità della prima pagina, presenta un difetto metodologico critico quando viene mediato su insiemi di query eterogenee:

- Se per una query specifica q_A esistono solo $R_A = 2$ documenti rilevanti nell'intera collezione, il valore massimo teoricamente raggiungibile per $P@10(q_A)$ è pari a $\frac{2}{10} = 0.20$, anche qualora il motore collochi perfettamente entrambi i documenti ai primi due ranghi!
- Calcolando la media aritmetica di $P@10$ tra query, il motore viene ingiustamente penalizzato dalla rarità intrinseca dei documenti rilevanti di alcune query, e non da un errore dell'algoritmo di ranking.

In modo analogo, la metrica complementare $Recall@k$ ($R@k$) calcola la frazione dei documenti rilevanti complessivi reperiti entro il rango k :

$$Recall@k = \frac{1}{R} \sum_{i=1}^k rel_i \quad (1.7)$$

1.7.2 Average Precision (AP)

L'**Average Precision** (AP) sintetizza l'intera curva Precision-Recall in un singolo indicatore scalare per una specifica query, valutando la precisione istantanea in corrispondenza di ogni posizione in cui il richiamo aumenta.

Definizione 1.7.2 (Average Precision). Dato un ordinamento di N documenti per una query con R documenti rilevanti totali presenti nella collezione, l'AP è definita come:

$$AP = \frac{1}{R} \sum_{k=1}^N P@k \cdot rel_k \quad (1.8)$$

Proprietà 1.7.1 (Interpretazione Geometrica e Ruolo di $1/R$). *L’Average Precision possiede una chiara interpretazione analitica:*

1. *Ogni volta che si incontra un documento rilevante al rango k , il richiamo compie un gradino discreto di ampiezza $\Delta\text{Recall} = \frac{1}{R}$. L’AP calcola la somma delle aree dei rettangoli $P@k \cdot \Delta\text{Recall}$, rappresentando l’**approssimazione numerica dell’Area Sotto la Curva Precision-Recall (AUC-PR)**.*
2. *La normalizzazione per R (il numero totale di documenti rilevanti nella collezione) garantisce che i documenti rilevanti non recuperati entro i primi N ranghi contribuiscano implicitamente con precisione pari a 0, penalizzando severamente i sistemi affetti da scarso richiamo.*

	Ordinamento del Motore di Ricerca →				
Posizione (Rango):	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$
Documento:	Rel	NonRel	Rel	NonRel	Rel
Precision@k:	1.00	—	0.67	—	0.60

Figura 1.4: Rappresentazione schematica del calcolo di Average Precision su 5 documenti con $R = 3$ rilevanti totali.

Esempio 1.7.2 (Calcolo di Base con $R = 3$ Rilevanti). Si consideri lo scenario di Figura 1.4 con $R = 3$ documenti rilevanti totali, posizionati ai ranghi $k_1 = 1$, $k_2 = 3$ e $k_3 = 5$:

$$P@1 = \frac{1}{1} = 1.00, \quad P@3 = \frac{2}{3} \approx 0.667, \quad P@5 = \frac{3}{5} = 0.60$$

L’Average Precision risulta:

$$AP = \frac{1}{3} \left(1.00 + \frac{2}{3} + \frac{3}{5} \right) = \frac{1}{3} (2.267) \approx 0.756 \quad (75.6\%) \quad (1.9)$$

Esempio 1.7.3 (Confronto Comparativo tra Due Ordinamenti). Si consideri una query per la quale esistono $R = 6$ documenti rilevanti nella collezione. Due motori di ricerca alternativi restituiscono i seguenti primi 10 documenti:

- **Ranking #1:** Rilevanti concentrati nelle prime posizioni:

⟨Rel, NonRel, Rel, Rel, Rel, Rel, NonRel, NonRel, NonRel, Rel⟩

- **Ranking #2:** Rilevanti dispersi e ritardati:

⟨NonRel, Rel, NonRel, NonRel, Rel, Rel, Rel, NonRel, Rel, Rel⟩

La Tabella 1.3 traccia l'evoluzione rango per rango di Precisione e Richiamo per entrambi i sistemi.

Rango (k)	Ranking #1			Ranking #2		
	Esito	Recall	Precision	Esito	Recall	Precision
1	Rel	0.17	1.00	NonRel	0.00	—
2	NonRel	0.17	—	Rel	0.17	0.50
3	Rel	0.33	0.67	NonRel	0.17	—
4	Rel	0.50	0.75	NonRel	0.17	—
5	Rel	0.67	0.80	Rel	0.33	0.40
6	Rel	0.83	0.83	Rel	0.50	0.50
7	NonRel	0.83	—	Rel	0.67	0.57
8	NonRel	0.83	—	NonRel	0.67	—
9	NonRel	0.83	—	Rel	0.83	0.56
10	Rel	1.00	0.60	Rel	1.00	0.60
Average Precision (AP)			0.78	0.52		

Tabella 1.3: Calcolo comparativo dettagliato di Average Precision tra Ranking #1 e Ranking #2.

I rispettivi valori di AP risultano:

$$AP(\text{Ranking \#1}) = \frac{1.00 + 0.67 + 0.75 + 0.80 + 0.83 + 0.60}{6} = \frac{4.65}{6} \approx \mathbf{0.78}$$

$$AP(\text{Ranking \#2}) = \frac{0.50 + 0.40 + 0.50 + 0.57 + 0.56 + 0.60}{6} = \frac{3.13}{6} \approx \mathbf{0.52}$$

Sebbene entrambi i motori raggiungano un richiamo finale identico ($R = 100\%$ sui 10 documenti), Ranking #1 ottiene un punteggio nettamente superiore (0.78 vs 0.52) poiché premia la densità di documenti rilevanti nella parte alta della graduatoria.

1.7.3 Mean Average Precision (MAP)

Quando si valuta un motore di ricerca su un banco di prova comprendente $|\mathcal{Q}|$ bisogni informativi distinti, si calcola la **Mean Average Precision** (MAP) come media aritmetica dei valori di AP:

Definizione 1.7.3 (Mean Average Precision).

$$MAP = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} AP(q) \quad (1.10)$$

Proprietà 1.7.2 (Macro-Averaging vs. Micro-Averaging). *Il MAP adotta formalmente una procedura di **Macro-Averaging**: ciascuna query possiede un peso identico pari a $1/|\mathcal{Q}|$.*

- *Se si adottasse il Micro-Averaging (ovvero sommare tutte le precisioni istantanee e dividere per il numero totale di documenti rilevanti dell'intera collezione), la misura verrebbe totalmente monopolizzata da pochissime query iper-generiche (ad es. "film" o "notizie", aventi decine di migliaia di documenti rilevanti), mascherando il fallimento del motore sulle query rare e specifiche (long-tail).*
- *Il Macro-Averaging garantisce che ogni singola esigenza informativa dell'utente riceva pari dignità nella valutazione finale.*

Esempio 1.7.4 (Calcolo di MAP su Due Query). Si consideri la valutazione congiunta di due query q_1 e q_2 :

- **Query 1 ($R_1 = 5$ rilevanti totali):** Il sistema restituisce 5 rilevanti ai ranghi $k \in \{1, 3, 6, 9, 10\}$ con precisioni istantanee pari a 1.00, 0.67, 0.50, 0.44 e 0.50:

$$AP(q_1) = \frac{1.00 + 0.67 + 0.50 + 0.44 + 0.50}{5} = \frac{3.11}{5} \approx \mathbf{0.62}$$

- **Query 2 ($R_2 = 3$ rilevanti totali):** Il sistema restituisce i 3 rilevanti ai ranghi $k \in \{2, 5, 7\}$ con precisioni istantanee pari a 0.50, 0.40 e 0.43:

$$AP(q_2) = \frac{0.50 + 0.40 + 0.43}{3} = \frac{1.33}{3} \approx \mathbf{0.44}$$

Il valore di Mean Average Precision del motore sui due compiti ammonta a:

$$MAP = \frac{AP(q_1) + AP(q_2)}{2} = \frac{0.62 + 0.44}{2} = \mathbf{0.53} \quad (1.11)$$

1.7.4 Metriche Troncate per il Web e Recommender Systems: AP@k e MAP@k

Nei moderni motori di ricerca su Web e nei sistemi di raccomandazione, calcolare l'AP sull'intera collezione risulta impraticabile (perché il numero totale R di rilevanti non è noto a priori) e poco rilevante per l'esperienza utente. Si adotta pertanto la variante troncata a soglia fissa K (**MAP@K**):

Definizione 1.7.4 (AP@K e MAP@K). Fissata la soglia di taglio K (tipicamente $K = 10$ o $K = 20$), l’Average Precision at K per la query q_i è definita come:

$$AP@K(q_i) = \frac{1}{K_i} \sum_{k=1}^K P@k(q_i) \cdot \text{rel}(q_i, k) \quad (1.12)$$

dove il fattore di normalizzazione K_i è dato dal numero di documenti rilevanti presenti nei primi K posti:

$$K_i = \sum_{k=1}^K \text{rel}(q_i, k) \quad (\text{oppure } K_i = \min(R_i, K)) \quad (1.13)$$

La metrica globale **MAP@K** su un insieme di query $\mathcal{Q}$ corrisponde alla media:

$$MAP@K(\mathcal{Q}) = \frac{1}{|\mathcal{Q}|} \sum_{q_i \in \mathcal{Q}} AP@K(q_i) \quad (1.14)$$

Questa formulazione risolve brillantemente il limite di $P@K$: normalizzando per K_i anziché rigidamente per K , una query con soli 2 rilevanti che li colloca ai primi 2 posti ottiene $AP@10 = 1.0$ anziché essere artificialmente limitata a 0.20.

1.7.5 Mean Reciprocal Rank (MRR)

In scenari quali le ricerche di elementi noti (*Known-Item Search*), le query navigazionali (ad es. cercare l’home page di un ateneo) o il reperimento di risposte fattuali, l’utente è interessato unicamente a individuare la **prima risposta corretta**. Lo sforzo cognitivo e il tempo speso dall’utente sono inversamente proporzionali alla posizione occupata dal primo risultato rilevante:

Definizione 1.7.5 (Reciprocal Rank e MRR). Sia $\text{rank}_1(q)$ la posizione del primo documento rilevante restituito per la query q . Il **Reciprocal Rank** (RR) è dato da:

$$RR(q) = \frac{1}{\text{rank}_1(q)} \quad (1.15)$$

Il **Mean Reciprocal Rank** (MRR) è la media aritmetica su tutte le query:

$$MRR = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{\text{rank}_1(q)} \quad (1.16)$$

Se per una query il primo documento rilevante si trova al rango 1, il punteggio è 1.0; se scende al rango 2 diviene 0.5; se scende al rango 10 crolla a 0.1.

1.8 Oltre la Rilevanza Binaria: Rilevanza Graduata e NDCG

Nelle applicazioni Web reali, l'ipotesi che un documento sia puramente rilevante (1) o non rilevante (0) risulta insufficiente. Si consideri la seguente query reale:

Query: "information retrieval unipi"

Una tipica SERP commerciale restituisce risultati con livelli qualitativi marcatamente differenti:

1. **DidaWiki del Corso (Docente, CFU, Programma, Materiale didattico):** Rappresenta la pagina principale che soddisfa pienamente l'intento dell'utente $\implies$ **Altamente Rilevante** ($r = 3$).
2. **Portale Ufficiale UniPi Valutami (Scheda d'esame e requisiti istituzionali):** Pagina autorevole ma meno ricca di dispense didattiche $\implies$ **Rilevante** ($r = 2$).
3. **Brochure PDF del Corso di Laurea Magistrale:** Contiene solo una breve menzione del corso $\implies$ **Marginalmente Rilevante** ($r = 1$).
4. **Articoli di cronaca o pagine non correlate:** $\implies$ **Irrilevante** ($r = 0$).

Tutti i primi tre documenti sono in qualche misura attinenti, ma un utente sarà soddisfatto solo se la pagina più informativa compare in cima. Per catturare questa granularità, la letteratura adotta scale a più livelli (tipicamente $r \in \{0, 1, 2, 3\}$ o $\{0, \dots, 4\}$).

L'approccio del **Discounted Cumulative Gain** (DCG), introdotto da Järvelin e Kekäläinen [8], poggia su due assunzioni fondamentali:

1. I documenti altamente rilevanti apportano un'utilità (*gain*) di gran lunga superiore rispetto a documenti solo marginalmente rilevanti.
2. Minore è la posizione ordinale (rango) di un documento nella graduatoria, minore è la probabilità che l'utente lo esamini; pertanto, il guadagno deve essere progressivamente scontato (*discounted*) verso il basso.

1.8.1 Cumulative Gain (CG) e Discounted Cumulative Gain (DCG)

Definizione 1.8.1 (Cumulative Gain a Rango p). Dato il vettore delle rilevanze graduate $\mathbf{r} = [r_1, r_2, \dots, r_p]$, il Cumulative Gain è la somma lineare dei guadagni:

$$CG_p = \sum_{i=1}^p r_i \quad (1.17)$$

Il CG_p è insensibile alla posizione: scambiare un documento altamente rilevante al rango 1 con uno al rango 10 non altera il valore di CG_p .

Definizione 1.8.2 (Discounted Cumulative Gain (Formulazione Standard)). Il DCG applica uno sconto logaritmico basato sul rango della posizione i :

$$DCG_p = r_1 + \sum_{i=2}^p \frac{r_i}{\log_2(i)} \quad (1.18)$$

Significato Comportamentale dello Sconto Logaritmico

Lo sconto $\frac{1}{\log_2(i)}$ riflette il decadimento esponenziale dell'attenzione visiva dell'utente secondo il modello di lettura a cascata:

- Al rango $i = 4$, il fattore di sconto è $\frac{1}{\log_2(4)} = \frac{1}{2}$: il valore informativo percepito si dimezza rispetto alla prima posizione.
- Al rango $i = 8$, il fattore è $\frac{1}{\log_2(8)} = \frac{1}{3}$.
- Al rango $i = 16$, il fattore scende a $\frac{1}{\log_2(16)} = \frac{1}{4}$.

Nei motori di ricerca commerciali (Google, Bing) si impiega frequentemente la **formulazione esponenziale**:

$$DCG_p^{\text{exp}} = \sum_{i=1}^p \frac{2^{r_i} - 1}{\log_2(i + 1)} \quad (1.19)$$

Chiarimento sulla Notazione degli Indici

Si noti la differenza nell'argomento del logaritmo:

- Nell'Equazione (1.18) il primo termine r_1 viene estratto per evitare $\log_2(1) = 0$ al denominatore, mentre la sommatoria parte da $i = 2$ con $\log_2(i)$.
- Nell'Equazione (1.19) la sommatoria include direttamente il rango $i = 1$ adottando $\log_2(i + 1)$ al denominatore ($\log_2(1 + 1) = \log_2(2) = 1$).
- Inoltre, il numeratore $2^{r_i} - 1$ enfatizza in modo non lineare i documenti di massima rilevanza ($r = 3 \implies 2^3 - 1 = 7$, mentre $r = 1 \implies 2^1 - 1 = 1$), penalizzando aspramente il loro mancato posizionamento nei primi ranghi.

1.8.2 Normalized Discounted Cumulative Gain (NDCG)

Poiché query differenti presentano un numero variabile di documenti rilevanti, il valore grezzo di DCG_p non è confrontabile tra query diverse. Si ricorre quindi all'**Ideal DCG** ($IDCG_p$), ottenuto calcolando il DCG della graduatoria ideale che ordina tutti i documenti per rilevanza decrescente:

$$NDCG_p = \frac{DCG_p}{IDCG_p} \in [0, 1] \quad (1.20)$$

1.8.3 Esempio Numerico Completo: Calcolo Passo-Passo

Si consideri la seguente graduatoria di 10 documenti, valutati su scala 0–3:

$$\mathbf{r} = [3, 2, 3, 0, 0, 1, 2, 2, 3, 0]$$

La sequenza ideale ottenuta ordinando per rilevanza non crescente è:

$$\mathbf{r}^* = [3, 3, 3, 2, 2, 2, 1, 0, 0, 0]$$

La Tabella 1.4 riporta l'evoluzione dettagliata di ogni singolo termine posizionale.

i	r_i	Sconto	Guadagno Scontato	$DCG@i$	r_i^*	Guadagno Ideale	$IDCG@i$	$NDCG@i$
1	3	1.000	3.00	3.00	3	3.00	3.00	1.00
2	2	$\log_2(2) = 1.000$	2.00	5.00	3	3.00	6.00	0.83
3	3	$\log_2(3) = 1.585$	1.89	6.89	3	1.89	7.89	0.87
4	0	$\log_2(4) = 2.000$	0.00	6.89	2	1.00	8.89	0.76
5	0	$\log_2(5) = 2.322$	0.00	6.89	2	0.86	9.75	0.71
6	1	$\log_2(6) = 2.585$	0.39	7.28	2	0.77	10.52	0.69
7	2	$\log_2(7) = 2.807$	0.71	7.99	1	0.36	10.88	0.73
8	2	$\log_2(8) = 3.000$	0.67	8.66	0	0.00	10.88	0.80
9	3	$\log_2(9) = 3.170$	0.95	9.61	0	0.00	10.88	0.88
10	0	$\log_2(10) = 3.322$	0.00	9.61	0	0.00	10.88	0.88

Tabella 1.4: Tracciamento dettagliato del calcolo di $DCG@i$, $IDCG@i$ e $NDCG@i$ per $i = 1, \dots, 10$.

1.8.4 Confronto tra Funzioni di Ranking Alternative

Si consideri il caso studio di 4 documenti $\{d_1, d_2, d_3, d_4\}$ con gradi di rilevanza:

$$r(d_1) = 0, \quad r(d_2) = 1, \quad r(d_3) = 2, \quad r(d_4) = 2$$

- **Ground Truth (GT):** Ordinamento $\langle d_4, d_3, d_2, d_1 \rangle$ con vettori di rilevanza $[2, 2, 1, 0]$:

$$IDCG_4 = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} + 0 = 2 + 2 + 0.6309 = 4.6309$$

$$NDCG_{GT} = 1.00$$

- **Ranking Function 1 (RF_1):** Ordinamento $\langle d_3, d_4, d_2, d_1 \rangle$ con rilevanze $[2, 2, 1, 0]$. Poiché i primi due documenti possiedono il medesimo grado ($r = 2$), l'inversione non produce alcuna penalità posizionale:

$$DCG_{RF_1} = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} = 4.6309$$

$$NDCG_{RF_1} = \frac{4.6309}{4.6309} = 1.00$$

- **Ranking Function 2 (RF_2):** Ordinamento $\langle d_3, d_2, d_4, d_1 \rangle$ con rilevanze $[2, 1, 2, 0]$. In questo caso, il documento d_2 ($r = 1$) scavalca erroneamente d_4 ($r = 2$):

$$DCG_{RF_2} = 2 + \frac{1}{\log_2(2)} + \frac{2}{\log_2(3)} + 0 = 2 + 1 + \frac{2}{1.585} = 4.2619$$

$$NDCG_{RF_2} = \frac{DCG_{RF_2}}{IDCG_4} = \frac{4.2619}{4.6309} \approx 0.9203$$

Questo calcolo dimostra con precisione matematica come la metrica NDCG penalizzi le inversioni di rango proporzionalmente alla gravità dello scambio di rilevanza.

Capitolo 2

Efficienza Algoritmica sulle Architetture CPU Moderne

2.1 Il Ruolo dell'Efficienza nell'Information Retrieval

Lo sviluppo di sistemi di elaborazione delle informazioni si articola lungo due dimensioni complementari e spesso in tensione tra loro:

1. **Efficacia (*Effectiveness*):** massimizza la soddisfazione dell'utente finale (*user satisfaction*), concentrandosi sulla qualità semantica dei risultati, sul ranking e sull'elaborazione del linguaggio naturale, tipicamente con strumenti ad alto livello di astrazione (Python, framework di Machine Learning).
2. **Efficienza (*Efficiency*):** minimizza i costi operativi infrastrutturali, progettando algoritmi ottimizzati fino all'ultimo ciclo di clock della CPU e all'ultimo bit di memoria occupato.

Non è richiesto che una singola figura professionale padroneggi entrambi gli estremi; tuttavia chi sviluppa ad alto livello deve comprendere i vincoli fisici dell'hardware sottostante per non progettare soluzioni impraticabili, mentre chi ottimizza a basso livello deve preservare la struttura e la semantica dei dati per non comprometterne l'efficacia.

2.1.1 I Tre Pilastri dell'Efficienza nei Sistemi IR

Esperienza utente. Nei sistemi web su scala planetaria, la risposta a una query deve essere erogata entro una frazione infinitesima di secondo. L'utente

percepisce qualsiasi ritardo superiore a pochi decimi di secondo: se i risultati tardano o risultano irrilevanti, abbandona il servizio a favore della concorrenza. L'efficienza algoritmica non serve solo a rispondere più rapidamente, ma consente di eseguire algoritmi di ranking e filtraggio più sofisticati ed efficaci *entro il medesimo budget temporale prestabilito* (tipicamente un tetto massimo di 100–200 ms).

Scalabilità operativa. I sistemi moderni gestiscono miliardi di documenti, elementi multimediali, query e log generati ogni secondo. Il numero di macchine necessarie è approssimativamente:

$$\text{Numero di macchine} = \left\lceil \frac{\text{Volume totale dei dati}}{\text{Capacità operativa del singolo nodo}} \right\rceil.$$

Se il volume dei dati raddoppia, un sistema ben progettato richiede al più il doppio delle macchine (scalabilità lineare); un algoritmo inefficiente può invece richiedere una crescita superlineare dei costi (ad es. quadruplicare i server), insostenibile su larga scala. Per rispettare le latenze richieste, i dati attivi devono risiedere in RAM ed evitare gli accessi a disco: ciò richiede algoritmi efficienti non solo in tempo ma anche in spazio, tramite tecniche di compressione. Inoltre, poiché i dati vengono replicati su più macchine per tolleranza ai guasti (*fault tolerance*), ogni inefficienza di memoria viene amplificata dal fattore di replica dell'infrastruttura.

Sostenibilità ambientale ed economica. La riduzione del numero di macchine fisiche abbatte direttamente il consumo energetico dei data center. I server ad alta densità dissipano ingenti quantità di calore e gran parte dell'energia è consumata dagli impianti di raffreddamento; per questo molti data center su larga scala vengono collocati in regioni fredde o vicino a corsi d'acqua (*free cooling*). Un risparmio computazionale marginale, anche solo del 3%, genera risparmi economici rilevanti e riduce l'impronta di carbonio.

2.2 La Gerarchia di Memoria e i Principi di Località

2.2.1 Il Divario di Latenza tra CPU e Memoria

Nel modello teorico computazionale classico (macchina RAM astratta) l'accesso a qualunque locazione di memoria ha costo unitario costante, $\mathcal{O}(1)$. Nelle macchine fisiche reali le unità aritmetiche della CPU operano a velocità

enormemente superiori rispetto alla memoria principale (DRAM), ed esiste una gerarchia di livelli intermedi (le *cache*) che approssima l'accesso rapido per i dati usati di frequente.

Livello hardware	Latenza tipica	Cicli CPU	Capacità tipica
Istruzione CPU (ALU)	0.5–5 ns	1–2	registri interni
L1I / L1D Cache	0.5 ns	≈ 1	32 KB per core
L2 Cache	1.5 ns	$\approx 3\text{--}4$	256 KB (fino a 2 MB)
L3 Cache (LLC)	5.0 ns	$\approx 10\text{--}15$	16 MB (fino a 64 MB+)
RAM principale (DRAM)	80.0 ns	$\approx 160\text{--}200+$	64 GB+
Disco / Storage	500 000 ns (0.5 ms)	milioni	8 TB+

Tabella 2.1: Confronto quantitativo di latenza e capacità tra i livelli della gerarchia di memoria.

Definizione 2.2.1 (Algoritmo *Memory-Bound*). Un algoritmo che accede alla RAM centrale in modo disordinato paga ≈ 80 ns per ogni accesso, intervallo in cui la CPU potrebbe eseguire centinaia di istruzioni. L'algoritmo si dice **memory-bound**: il tempo di esecuzione è dominato dall'attesa del trasferimento dati dalla DRAM e la CPU trascorre la maggior parte del tempo in stallo, rendendo irrilevante l'ottimizzazione del numero di operazioni aritmetiche.

2.2.2 Architettura Fisica sul Chip

La cache L1 è integrata a ridosso dell'unità di decodifica e delle unità di esecuzione; per evitare conflitti di bus è fisicamente suddivisa in **L1I** (*Instruction Cache*, contiene il codice macchina prelevato per l'esecuzione) e **L1D** (*Data Cache*, contiene gli operandi delle istruzioni di `load/store`). La cache L2 è integrata nel singolo core e funge da cuscinetto a bassa latenza; la cache L3 è esterna ai singoli core ma interna al package ed è condivisa tra tutti i core della macchina.

La Cache Line come Unità Minima di Trasferimento

La RAM e la gerarchia di cache non trasferiscono mai singoli byte o singole variabili isolate: l'unità minima di trasferimento hardware è la **cache line**, tipicamente $B = 64$ byte. Se un programma necessita di un intero a 64 bit (8 byte) situato all'indirizzo A , l'hardware carica forzatamente l'intero blocco allineato

$$[\lfloor A/64 \rfloor \times 64, \lfloor A/64 \rfloor \times 64 + 63].$$

Se il programma legge quell'unico valore e poi salta a un indirizzo casuale, i restanti 56 byte (87.5% del traffico di memoria) vengono sprecati.

2.2.3 Località Temporale e Spaziale

Un programma sfrutta efficacemente la gerarchia di memoria solo se rispetta almeno una delle due forme di località:

- **Località temporale:** quando una data locazione di memoria viene acceduta, è altamente probabile che la medesima locazione venga acceduta nuovamente nell'immediato futuro. Le politiche di rimpiazzo delle cache (varianti di **LRU**, *Least Recently Used*) scartano i dati meno recentemente utilizzati per preservare le locazioni accedute di frequente.
- **Località spaziale:** quando una data locazione di memoria viene acceduta, è altamente probabile che locazioni contigue o vicine vengano accedute nell'immediato futuro. Poiché l'hardware trasferisce l'intera cache line, l'accesso a elementi contigui ammortizza il costo di caricamento su tutti gli elementi contenuti al suo interno.

2.2.4 Hardware Prefetcher

I processori integrano circuiti dedicati, gli **hardware prefetcher**, il cui compito è anticipare le richieste di memoria dell'applicazione:

- **Stream prefetcher:** se un programma scansiona linearmente la memoria da sinistra verso destra ($A, A + 64, A + 128, \dots$), il circuito rileva il pattern sequenziale e ordina il caricamento preventivo delle cache line successive in L1/L2 prima ancora che il programma emetta l'istruzione di lettura, mascherando così la latenza.

- **Stride prefetcher:** intercetta accessi a passo costante (*stride*), ad esempio scansioni che campionano la memoria a intervalli fissi; riconosciuto il gradiente costante, carica selettivamente le cache line corrispondenti al passo di salto.
- **Pattern casuali:** di fronte a salti casuali e non deterministici (*pointer chasing*), nessun hardware prefetcher è in grado di anticipare gli accessi, causando lo stallo completo della pipeline a ogni lettura.

2.3 Caso di Studio: Scansione Sequenziale contro *Pointer Chasing*

Per dimostrare sperimentalmente l'impatto della località e del prefetching si confrontano due funzioni Rust operanti su un vettore v di dimensione N . Il vettore di ingresso rappresenta una **permutazione casuale con un singolo ciclo** $\pi \in S_N$.

Definizione 2.3.1 (Permutazione con un singolo ciclo). Una permutazione π dell'insieme $\{0, 1, \dots, N - 1\}$ contiene un singolo ciclo se e solo se, partendo dall'indice 0, la catena $k_{t+1} = \pi(k_t)$ visita ogni posizione dell'array esattamente una volta prima di tornare a 0:

$$0 \rightarrow \pi(0) \rightarrow \pi(\pi(0)) \rightarrow \dots \rightarrow \pi^{N-1}(0) \rightarrow 0,$$

senza cicli isolati o sottocicli disgiunti.

Entrambe le funzioni calcolano la sommatoria di tutti gli elementi del vettore, $\sum_{i=0}^{N-1} v[i] = \frac{(N-1)N}{2}$.

Listing 2.1: Scansione sequenziale lineare.

```

1 pub fn sum(v: &[usize]) -> usize {
2     let mut sum = 0;
3     for &value in v.iter() {
4         sum += value;
5     }
6     sum
7 }
```

Listing 2.2: Scansione per salti casuali (*pointer chasing*).

```

1 pub fn sum_by_jumping(v: &[usize]) -> usize {
2     let mut sum = 0;
```

```

3   let mut k = 0;
4   for _ in 0..v.len() {
5       sum += v[k];
6       k = v[k]; // il valore letto diventa l'indice
                    della prossima iterazione
7   }
8   sum
9 }

```

Nel modello teorico RAM entrambe le funzioni eseguono N iterazioni con un'operazione di somma per passo: hanno cioè identica complessità asintotica $\mathcal{O}(N)$. Sulla macchina fisica, misurando il tempo medio per singolo elemento al variare della dimensione dell'array, emergono però divergenze marcate.

N	Dim.	Livello cache	Tempo per elem.		Rallentamento
			sum()	jumping()	
256	2 KiB	L1 (32 KB)	0.1 ns	1.3 ns	13×
8 192	64 KiB	L2 (256 KB)	0.1 ns	2.5 ns	25×
262 144	2 MiB	L3 (16 MB)	0.2 ns	13.4 ns	67×
16 777 216	128 MiB	RAM (DRAM)	0.4 ns	84.9 ns	212×

Tabella 2.2: Tempo medio per elemento di `sum()` e `sum_by_jumping()` al variare della dimensione dell'array.

2.3.1 Analisi delle Cause Fisiche

Perché `sum()` varia solo da 0.1 a 0.4 ns? Con interi `usize` a 64 bit (8 byte), in una cache line da 64 byte alloggiano $64/8 = 8$ elementi contigui: si ha piena località spaziale. L'accesso a una nuova linea comporta un miss iniziale, ma i successivi 7 accessi avvengono a costo pressoché nullo da cache, e lo stream prefetcher carica le linee successive in anticipo rispetto all'esecuzione. Indicando con T_{comp} il tempo di calcolo e con T_{load} il tempo di trasferimento ammortizzato, il costo osservato per elemento segue

$$\text{Latenza osservata} \approx \max(T_{\text{comp}}, T_{\text{load_ammortizzato}}),$$

quasi interamente mascherato dal calcolo; l'aumento da 0.1 a 0.4 ns su 128 MiB è dovuto unicamente alla saturazione della banda del bus verso la RAM.

Perché `sum_by_jumping()` arriva a 84.9 ns? Poiché la permutazione è casuale, l'indirizzo $k_{t+1} = v[k_t]$ punta a una locazione arbitraria: la probabilità che cada nella stessa cache line è trascurabile, quindi per ogni elemento letto

da 8 byte i restanti 56 byte trasferiti vengono scartati (assenza totale di località spaziale). Non essendovi passo lineare o costante, il prefetching è inibito. Inoltre la CPU deve conoscere l'indice della lettura successiva prima di poterla emettere: si tratta di una **dipendenza seriale** che impedisce ogni sovrapposizione. Quando la dimensione dell'array supera la capienza della L3 (128 MiB $\gg$ 16 MB), ogni accesso diventa un cache miss globale verso la DRAM:

Tempo per elemento $\approx$ latenza di accesso fisico alla DRAM $\approx$ 80–85 ns, valore che coincide esattamente con i 84.9 ns misurati.

Esempio 2.3.1 (Ammortizzare il costo del *pointer chasing* sfruttando la cache line). Se il salto avviene tra blocchi di 64 byte (8 interi a 64 bit), ma tutti gli elementi del blocco vengono sommati prima del salto successivo, il costo di un cache miss verso la RAM ($\approx$ 85 ns) viene sostenuto una sola volta ogni 8 operazioni di somma:

```

1  pub fn sum_by_jumping_cache_lines(v: &[usize]) -> usize
2      {
3          let mut sum = 0;
4          let mut k = 0;
5          let num_blocks = v.len() / 8;
6          for _ in 0..num_blocks {
7              let base = k * 8;
8              // sfrutta appieno la cache line gia' caricata
9              // in memoria
10             sum += v[base] + v[base + 1] + v[base + 2] + v[
11                 base + 3]
12                 + v[base + 4] + v[base + 5] + v[base + 6];
13             k = v[base + 7] % num_blocks; // puntatore al
14             // prossimo salto
15         }
16         sum
17     }

```

La latenza viene ammortizzata su 8 elementi, portando il tempo medio atteso per valore a

$$\text{Tempo medio per valore} \approx \frac{85 \text{ ns}}{8} \approx 10.6 \text{ ns},$$

un risultato intermedio tra la scansione lineare e il salto casuale elemento per elemento.

Nota 2.3.1 (Eliminare il rallentamento residuo di `sum()`). Il lieve rallentamento da 0.1 a 0.4 ns osservato in `sum()` si verifica perché $T_{\text{comp}} \approx 0.1 \text{ ns}$ è

inferiore al tempo necessario ad alimentare il flusso dati dalla RAM su vettori molto grandi (regime memory-bound). Aumentando la complessità computazionale del corpo del ciclo (ad es. moltiplicazioni o funzioni non banali su ciascun elemento) si ottiene $T_{\text{comp}} \gg T_{\text{load_RAM}}$: la relazione $\max(T_{\text{comp}}, T_{\text{load}})$ diventa costantemente pari a T_{comp} , il regime diventa *CPU-bound* e la curva dei tempi risulta perfettamente piatta al variare di N .

2.4 Forme di Parallelismo e Architettura della Pipeline

Le moderne CPU sfruttano molteplici forme di parallelismo: il **multi-threading/multi-core** (più core fisici indipendenti sullo stesso processore), il **pipelining** (suddivisione dell'esecuzione dell'istruzione in stadi operanti simultaneamente su istruzioni contigue) e l'esecuzione **superscalare** (unità di calcolo multiple nello stesso core per elaborare più istruzioni indipendenti per ciclo).

Parallelismo Triviale nei Motori di Ricerca

Nei motori di ricerca non è necessario sincronizzare più core per rispondere a una singola query: si adotta il **parallelismo triviale** (*embarrassingly parallel*), in cui, data la presenza di migliaia di utenti simultanei, ogni thread su un singolo core serve in modo completamente isolato una specifica query. Non essendovi condivisione di stato, si azzerano le contese sui lock e i problemi di coerenza della cache.

2.4.1 Anatomia del Pipelining a Cinque Stadi

Il pipelining scompone l'esecuzione di un'istruzione in stadi sequenziali indipendenti che, in un'architettura standard, sono cinque:

1. **IF** (*Instruction Fetch*): prelievo dell'istruzione in codice macchina dalla memoria (L1I) tramite il *program counter*.
2. **ID** (*Instruction Decode*): decodifica dell'istruzione in micro-operazioni (μ -ops) e lettura dei registri sorgente.
3. **EX** (*Execute*): esecuzione del calcolo aritmetico-logico nelle ALU o calcolo dell'indirizzo effettivo di memoria.
4. **MEM**: accesso al sottosistema di memoria (L1D) per istruzioni di caricamento o scrittura (LOAD/STORE).

5. **WB** (*Write-Back*): scrittura del risultato nei registri architetturali e ritiro dell'istruzione.

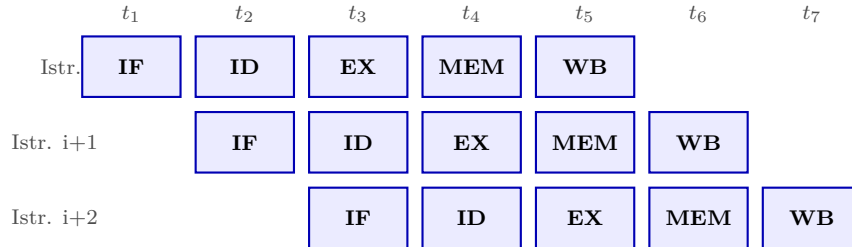


Figura 2.1: Sovrapposizione degli stadi della pipeline a 5 stadi su istruzioni consecutive.

2.4.2 Latenza, Throughput e Profondità della Pipeline

Sia k il numero di stadi della pipeline e τ il periodo del ciclo di clock. La **latenza** L è il tempo totale richiesto da una singola istruzione per attraversare l'intera pipeline, $L = k \cdot \tau$: nelle CPU moderne la latenza per un'operazione semplice varia tipicamente tra 10 e 20 cicli di clock. Il **throughput** W è il numero di istruzioni completate per unità di tempo e, a regime, $W = 1/\tau$ istruzioni al secondo, poiché a ogni ciclo un'istruzione esce dalla pipeline. Il periodo τ è imposto dal ritardo dello stadio più lento della catena circuitale,

$$\tau \geq \max_{1 \leq i \leq k} (t_{\text{stadio}_i}).$$

Per innalzare la frequenza di clock (ridurre τ) i progettisti aumentano la profondità k della pipeline (fino a 15–20 o più stadi nelle CPU moderne), implementando ciascuno stadio con circuiteria più semplice e veloce; di contro, l'aumento di k dilata la penalità in caso di svuotamento forzato della pipeline.

2.5 Esecuzione Superscalare, Interleaving e Conflitti Strutturali

Un processore si definisce **superscalare** quando possiede l'abilità architetturale di avviare e completare più di un'istruzione per ciclo di clock ($IPC > 1$, *Instructions Per Cycle*) all'interno dello stesso core, integrando unità logico-aritmetiche multiple (ALU per interi, moltiplicatori, unità di shift, unità di load/store) servite da porte di emissione parallele (*execution ports*). Architetture reali come Intel Haswell dispongono ad esempio di una coda di decodifica

a 56 voci, un *reorder buffer* a 192 voci, una stazione di prenotazione unificata a 60 voci e 8 porte di esecuzione parallele, ciascuna specializzata per classi di operazioni (ALU intere, load/store, unità vettoriali, branch, ecc.).

2.5.1 L'Esperimento dell'*Interleaving* Aritmetico

Si sottopongono a benchmark quattro funzioni Rust che operano su un vettore di interi a 64 bit `&[u64]`, accumulando un numero crescente di operazioni indipendenti per iterazione.

Listing 2.3: Da un'unica riduzione a quattro riduzioni indipendenti per iterazione.

```
1 pub fn do_stuff(v: &[u64]) -> u64 {
2     let mut prod = 1;
3     for &value in v.iter() {
4         prod *= value;
5     }
6     prod
7 }
8
9 pub fn do_more_stuff(v: &[u64]) -> (u64, u64) {
10     let mut prod = 1;
11     let mut sum = 0;
12     for &value in v.iter() {
13         prod *= value;
14         sum += value;
15     }
16     (prod, sum)
17 }
18
19 pub fn do_more_stuff_2(v: &[u64]) -> (u64, u64, u64) {
20     let mut prod = 1;
21     let mut sum = 0;
22     let mut xor = 0;
23     for &value in v.iter() {
24         prod *= value;
25         sum += value;
26         xor ^= value;
27     }
28     (prod, sum, xor)
29 }
30
31 #[inline]
```

```

32 pub fn do_more_stuff_3(v: &[u64]) -> (u64, u64, u64, u64)
    {
33     let mut prod = 1;
34     let mut sum = 0;
35     let mut xor = 0;
36     let mut or = 0;
37     for &value in v.iter() {
38         prod *= value;
39         sum += value;
40         xor ^= value;
41         or |= value;
42     }
43     (prod, sum, xor, or)
44 }

```

Funzione	Tempo per iterazione
do_stuff() (1 operazione)	0.4 ns
do_more_stuff() (2 operazioni)	0.4 ns
do_more_stuff_2() (3 operazioni)	0.4 ns
do_more_stuff_3() (4 operazioni)	0.5 ns

Tabella 2.3: Tempo per iterazione al crescere del numero di riduzioni indipendenti accumulate.

2.5.2 Sovrapposizione delle Istruzioni e Saturazione delle Porte

Da una a tre operazioni il tempo resta invariato a 0.4 ns perché le variabili di accumulo non presentano dipendenze reciproche: $\text{prod}_{t+1} = \text{prod}_t \cdot v[i]$, $\text{sum}_{t+1} = \text{sum}_t + v[i]$, $\text{xor}_{t+1} = \text{xor}_t \oplus v[i]$. Il core superscalare invia contemporaneamente la moltiplicazione, l'addizione e lo XOR a tre porte di esecuzione distinte: le tre operazioni vengono completate nel medesimo ciclo di clock, e il lavoro aggiuntivo è svolto a costo nullo.

Con la quarta operazione (**or**) il tempo sale a 0.5 ns anziché raddoppiare a 0.8 ns: si tratta di uno **structural hazard**, poiché le unità fisiche disponibili per calcoli logico-aritmetici in parallelo in un singolo ciclo sono sature (solo tre istruzioni di quel tipo possono essere emesse simultaneamente), e una delle quattro operazioni deve attendere. Tuttavia la CPU *out-of-order* non processa le iterazioni come compartimenti stagni: srotola implicitamente la finestra di istruzioni raggruppandole a blocchi di tre, assorbendo l'operazione in eccesso nei cicli successivi e saturando le capacità residue delle porte. L'overhead

reale corrisponde quindi a una frazione marginale di ciclo ($\Delta t = +0.1 \text{ ns}$) anziché a un ciclo intero aggiuntivo.

2.6 Data Hazard e Dipendenze Seriali

Definizione 2.6.1 (Data Hazard). Si verifica un **data hazard** quando un'istruzione necessita di un operando non ancora disponibile poiché in fase di calcolo da parte di un'istruzione precedente. Corrisponde a una dipendenza sui dati di tipo **RAW** (*Read-After-Write*):

$$\text{Inst}_A : R_1 \leftarrow R_2 + R_3 \quad \Longrightarrow \quad \text{Inst}_B : R_4 \leftarrow R_1 \times R_5,$$

dove Inst_B deve rimanere in stallo nella pipeline finché Inst_A non conclude il calcolo.

Si consideri un codice che esegue lo stesso numero di operazioni per iterazione della precedente `do_more_stuff_3`, ma concatenando i dati in una catena stretta:

Listing 2.4: Catena di dipendenza seriale RAW tra quattro operazioni.

```

1 pub fn do_more_stuff_4(v: &[u64]) -> (u64, u64, u64) {
2     let mut prod = 1;
3     let mut sum = 0;
4     let mut xor = 0;
5     for &value in v.iter() {
6         sum += value; // 1. calcola sum
7         prod *= sum; // 2. dipende strettamente da sum
8         xor ^= prod; // 3. dipende strettamente da
9                       prod
10        sum += xor; // 4. dipende da xor e
11                   sovrascrive sum per l'iterazione successiva!
12    }
13    (prod, sum, xor)
14 }
```

A causa della catena RAW, le unità di esecuzione non possono avviare alcun calcolo in parallelo:

$$\text{Latenza iterazione} = t_{\text{add}} + t_{\text{mul}} + t_{\text{xor}} + t_{\text{add_feedback}}.$$

Poiché l'ultima operazione aggiorna `sum` e la prima istruzione dell'iterazione successiva richiede il nuovo valore di `sum`, la CPU *out-of-order* non può avviare l'iterazione $t + 1$ in anticipo: le unità superscalari restano inutilizzate.

Sperimentalmente, contro i 0.5 ns di `do_more_stuff_3` (operazioni indipendenti), `do_more_stuff_4` misura 1.7 ns per iterazione ($\approx 3.5\times$ più lento), pur eseguendo lo stesso numero di operazioni aritmetiche.

2.6.1 Soluzione Algoritmica: *Interleaving* di Vettori Indipendenti

Quando occorre eseguire tale calcolo su due distinti vettori `V` e `West`, l'approccio sequenziale (chiamando `do_more_stuff_4` su `V` e poi su `West`) impiega un tempo complessivo pari a $2T$. Se invece si scrive un'unica funzione che elabora entrambi i vettori all'interno della stessa iterazione, alternando le due catene indipendenti:

Listing 2.5: Fusione (interleaving) di due catene RAW indipendenti tra loro.

```
1  for i in 0..n {
2      // catena per V (occupa le porte al ciclo k)
3      sum_v += v[i];
4      prod_v *= sum_v;
5      xor_v ^= prod_v;
6      sum_v += xor_v;
7      // catena per West, completamente indipendente da V!
8      sum_w += west[i];
9      prod_w *= sum_w;
10     xor_w ^= prod_w;
11     sum_w += xor_w;
12 }
```

il tempo di esecuzione misurato è identico a quello del solo vettore `V` ($\approx T$, anziché $2T$). Le operazioni su `West` non dipendono da quelle su `V`: mentre la catena di `V` è in attesa dei risultati delle ALU, le unità di calcolo libere processano la catena di `West`, saturando a costo nullo i tempi morti imposti dalle dipendenze sui dati.

2.7 Control Hazard, Esecuzione Speculativa e Branch Prediction

Definizione 2.7.1 (Control Hazard). Si verifica un **control hazard** quando la CPU non può determinare quale sia la successiva istruzione da prelevare dalla memoria (*instruction fetch*) a causa della presenza di un salto condizionato (`if`, `branch`).

La CPU non può fermarsi a ogni condizione: adotta pertanto l'**esecuzione speculativa**, ipotizzando tramite un *branch predictor* quale ramo verrà imboccato e prelevando speculativamente le istruzioni successive per alimentare la pipeline. Se l'esito della condizione si rivela discorde rispetto alla predizione, tutto il lavoro speculativo completato negli stadi intermedi deve essere annullato, con uno svuotamento completo della pipeline (*pipeline flush*) e il ricaricamento dal ramo corretto:

Penalità per mispredizione = 15–20 cicli di clock.

2.7.1 Il Paradosso della Condizione Casuale

Si introduce un filtro condizionale in un codice con catena di dipendenza, operando su un array di interi casuali uniformemente distribuiti in $[0, 99]$:

Listing 2.6: Ramo condizionale su valori casuali: la CPU non può predire l'esito.

```
1 pub fn do_more_stuff_5(v: &[u64]) -> (u64, u64, u64) {
2     let mut prod = 1;
3     let mut sum = 0;
4     let mut xor = 0;
5     for &value in v.iter() {
6         if value > 50 { // vera con p = 0.5 su valori
7             sum += value;
8             prod *= sum;
9             xor ^= prod;
10            sum += xor;
11        }
12    }
13    (prod, sum, xor)
14 }
```

Poiché la condizione è vera in media il 50% delle volte, ci si aspetterebbe intuitivamente un tempo pressoché dimezzato rispetto a `do_more_stuff_4` ($1.7/2 \approx 0.85$ ns). Il tempo misurato sale invece a 3.5 ns, più del doppio più lento della versione priva di condizione.

Il Costo Reale non è il Calcolo ma la Mispredizione

Con valori casuali il branch predictor ha accuratezza pari al lancio di una moneta, $P(\text{hit}) = P(\text{miss}) = 0.50$. Il costo medio per iterazione è allora

$$\begin{aligned}\text{Costo medio} &\approx 0.5 \times T_{\text{corpo}} + P(\text{miss}) \times \text{Penalità flush} \\ &\approx 0.5 \times 0.85 \text{ ns} + 0.5 \times 5.0 \text{ ns} \approx 2.9\text{--}3.5 \text{ ns},\end{aligned}$$

avendo assunto una penalità di 15–20 cicli ($\approx 4\text{--}5$ ns su una CPU a 4 GHz). Il costo dell'algoritmo non è più dettato dalle operazioni aritmetiche, ma è dominato dal tempo perso a svuotare e ricaricare la pipeline.

2.7.2 Verifica Sperimentale: Ordinamento dei Dati e Pattern Periodici

Per dimostrare che il rallentamento deriva esclusivamente dal comportamento del branch predictor, si esegue lo stesso identico codice modificando solo la disposizione dei dati nell'array.

Configurazione	Tempo / iter.	Comportamento del predittore
<code>do_more_stuff_4()</code> (nessun <code>if</code>)	1.7 ns	—
<code>do_more_stuff_5()</code> (dati casuali)	3.5 ns	Ramo imprevedibile, miss rate $\approx 50\%$
<code>do_more_stuff_5()</code> (dati ordinati)	1.2 ns	Predizione quasi perfetta, un solo miss
<code>do_more_stuff_5()</code> (dati alternati)	1.1 ns	Sequenza periodica, miss rate $\approx 0\%$

Tabella 2.4: Tempo per iterazione di `do_more_stuff_5()` al variare della disposizione dei dati.

Con **dati ordinati** (tutti gli elementi ≤ 50 all'inizio dell'array e tutti quelli > 50 alla fine) il predittore assume che il salto non sia preso per l'intera prima metà e che sia preso per la seconda: si verifica un unico misprediction al punto di transizione, e rimosso il costo dei flush il tempo scende a 1.2 ns, inferiore ai 1.7 ns della versione senza condizione poiché metà delle iterazioni non esegue il corpo dell'`if`. Con **dati alternati** secondo un pattern periodico (*falso, vero, falso, vero, ...*), i predittori integrano tabelle storiche dei pattern

(*branch history shift register*): identificata la periodicità, il predittore anticipa l'esito a ogni passo senza errori, annullando gli stalli.

2.7.3 Tecniche *Branchless* e il Salto Implicito dei Cicli `for`

Se i dati non sono ordinabili né predicibili, l'unica soluzione consiste nel rimuovere fisicamente il salto condizionale (*branchless code*), sostituendolo con espressioni algebriche o istruzioni condizionali assembly (*CMOV*). Ad esempio, invece di

```
1 if value > 50 {  
2     sum += value;  
3 }
```

si mappa la guardia logica su un valore binario scalare (0 o 1), eseguendo sempre l'operazione aritmetica:

```
1 let condition = (value > 50) as u64; // vale 1 se vero,  
    0 se falso  
2 sum += value * condition;           // somma 'value' se  
    vero, somma '0' se falso
```

Il programma compie sempre il calcolo matematico, ma il flusso di istruzioni è lineare: non vi sono salti, e le mispredizioni sono azzerate.

Nota 2.7.1 (Perché il salto del ciclo `for` non costituisce un problema). Ogni ciclo `for` si conclude a livello macchina con un salto condizionato all'inizio del blocco. Questo branch non degrada le prestazioni perché è altamente asimmetrico: per N iterazioni la condizione di permanenza nel ciclo risulta vera per $N - 1$ volte consecutive ed è falsa solo all'uscita. Il branch predictor impara immediatamente il pattern ("salto preso") e commette un unico errore di predizione al termine del ciclo, con un impatto del tutto trascurabile sul tempo complessivo.

2.8 Sintesi: dai Vincoli Hardware alle Scelte Algoritmiche

I fenomeni analizzati in questo capitolo – gerarchia di memoria e località, pipelining, esecuzione superscalare, *data hazard* e *control hazard* – non sono dettagli implementativi trascurabili, ma vincoli fisici che determinano il reale comportamento prestazionale degli algoritmi di Information Retrieval su larga scala. In particolare:

- le strutture dati (posting list, indici invertiti) vanno progettate privilegiando l'accesso sequenziale e contiguo, per sfruttare la località spaziale e l'hardware prefetching;
- quando possibile, conviene esprimere più riduzioni indipendenti all'interno dello stesso ciclo per saturare le porte di esecuzione superscalari a costo marginale;
- le catene di dipendenza strette (RAW) tra variabili di accumulo vanno identificate e, se possibile, disaccoppiate o intrecciate (*interleaving*) con calcoli indipendenti;
- i rami condizionali imprevedibili sui dati vanno eliminati con tecniche *branchless* o mitigati riordinando i dati per favorire la predizione.

La trattazione della vettorizzazione SIMD e degli strumenti di analisi del codice compilato (*Compiler Explorer*) è rimandata a un capitolo successivo.

Capitolo 3

Elaborazione del Linguaggio Naturale e Rappresentazione del Testo

3.1 Recap: Metriche di Valutazione per Rilevanza Graduata (CG, DCG, NDCG)

Nei sistemi di Information Retrieval reali, l'assunzione di rilevanza meramente binaria (documento rilevante o non rilevante) non riflette la varietà qualitativa dei risultati restituiti da un motore di ricerca. I giudizi di rilevanza vengono pertanto espressi su scale graduate (ad esempio su scala Likert da 0 a 3 o da 0 a 4, indicando livelli crescenti di pertinenza).

La valutazione di una graduatoria ordinata (*ranking*) di documenti alla posizione p si fonda su due principi cardine:

1. **Guadagno (*Gain*):** più in alto compaiono documenti a elevata rilevanza all'interno della SERP (*Search Engine Results Page*), maggiore è l'utilità percepita dall'utente.
2. **Sconto Posizionale (*Discounting*):** documenti rilevanti collocati in posizioni basse della classifica forniscono un'utilità progressivamente inferiore rispetto alla medesima rilevanza posizionata in cima, poiché l'attenzione dell'utente decresce monotonicamente scorrendo la lista.

3.1.1 Cumulative Gain (CG) e Discounted Cumulative Gain (DCG)

Dato un vettore di rilevanze graduate $\mathbf{r} = [r_1, r_2, \dots, r_p]$ per i primi p documenti restituiti per una query q :

Definizione 3.1.1 (Cumulative Gain a Rango p). Il **Cumulative Gain** (CG_p) somma linearmente i valori di rilevanza fino alla soglia p :

$$CG_p = \sum_{i=1}^p r_i. \quad (3.1)$$

Limite intrinseco: il CG_p è del tutto insensibile all'ordine: collocare un documento con $r_i = 3$ al primo o al decimo posto produce lo stesso valore di CG .

Per modellare il decadimento dell'attenzione dell'utente con il progredire della posizione i , si introduce una funzione di sconto logaritmico al rango:

Definizione 3.1.2 (Discounted Cumulative Gain a Rango p). Nella formulazione standard e nella variante a guadagno esponenziale ampiamente impiegata nei motori di ricerca industriali:

$$DCG_p = r_1 + \sum_{i=2}^p \frac{r_i}{\log_2(i)}, \quad (3.2)$$

$$DCG_p^{\text{exp}} = \sum_{i=1}^p \frac{2^{r_i} - 1}{\log_2(i + 1)}. \quad (3.3)$$

La formulazione esponenziale (3.3) enfatizza marcatamente il recupero di documenti a rilevanza massima nelle primissime posizioni della graduatoria.

3.1.2 Normalized Discounted Cumulative Gain (NDCG) e la Necessità della Normalizzazione

Il DCG grezzo presenta un limite strutturale fondamentale: è una misura strettamente **query-dependent**. Se per una specifica query q_A esistono nella collezione 100 documenti altamente rilevanti ($r = 3$), il valore assoluto di DCG ottenibile sarà intrinsecamente elevato. Se per un'altra query q_B esiste un solo documento rilevante in tutto il corpus, il valore massimo teorico di DCG sarà modesto. Di conseguenza, i valori grezzi di DCG non possono essere confrontati né mediati direttamente tra query eterogenee.

Per consentire la comparabilità e il calcolo della media globale delle prestazioni (*macro-averaging*) su un intero banco di valutazione (*benchmark*),

si definisce l'**Ideal DCG** ($IDCG_p$), calcolato sulla graduatoria ideale ottenuta ordinando tutti i documenti per rilevanza decrescente:

$$NDCG_p = \frac{DCG_p}{IDCG_p} \in [0, 1]. \quad (3.4)$$

In virtù della normalizzazione, un valore di $NDCG_p = 1.0$ certifica che il sistema ha ordinato i documenti esattamente nella sequenza ottima possibile fino alla profondità p .

3.2 Caratteristiche del Linguaggio Naturale e Legge di Zipf

3.2.1 Proprietà e Complessità del Linguaggio Naturale

L'elaborazione automatica del linguaggio naturale pone ostacoli computazionali intrinseci dovuti alla sua peculiare struttura:

- **Migliaia di simboli distinti:** i vocabolari reali comprendono centinaia di migliaia di parole uniche.
- **Sintassi gerarchica e complessa:** le frasi non sono semplici catene sequenziali, ma alberi sintattici caratterizzati da dipendenze a lungo raggio.
- **Semantica composizionale:** il significato complessivo di un enunciato deriva dal significato dei singoli costituenti e dall'ordine sintattico preciso con cui si combinano.
- **Ambiguità poliedrica:**
 - *Part-of-Speech (POS) ambiguity:* il medesimo termine funge da verbo o da sostantivo a seconda del contesto (es. *beat*: “*Listen to this cool beat*” [sostantivo] vs “*I’ll beat you at checkers!*” [verbo]).
 - *Word sense ambiguity (Polisemia e Omonimia):* un termine possiede più accezioni semantiche (es. *interest*: “*There is interest on this topic*” [attenzione] vs “*The bank raised the interest rate*” [tasso di interesse bancario]).
 - *Syntactic ambiguity:* proposizioni con molteplici alberi di derivazione plausibili (es. “*I shot an elephant in my pajamas*”, dove il sintagma preposizionale può legarsi sintatticamente al soggetto o all'oggetto).

- **Forte dipendenza dalla conoscenza del mondo (*Shared World Knowledge*):**
 - “*I’m faster than Lewis Hamilton*”: richiede la conoscenza ontologica pregressa che Lewis Hamilton sia un pilota di Formula 1 per disambiguare l’ambito sportivo della velocità.
 - “*Alexis and Kate are sisters*”: esprime una relazione di parentela simmetrica tra due entità.
 - “*Alexis and Kate are mothers*”: attribuisce la medesima proprietà a entrambe le persone, senza implicare alcuna relazione diretta tra loro.
 - “*My mother is younger than me*”: sintatticamente corretta, ma semanticamente e biologicamente impossibile.
- **Evoluzione temporale continua:** il lessico muta rapidamente per effetto del progresso tecnologico e di eventi socio-culturali con neologismi e slittamenti semantici (es. “*to google*”, “*crowdfunded*”, “*in the cloud*”, “*super spreader*”, “*wear a mask*”).

3.2.2 La Legge di Zipf e il Principio del Minimo Sforzo

In qualsiasi corpus testuale di linguaggio naturale, la frequenza empirica f di un termine è inversamente proporzionale al suo rango r nella tabella delle frequenze ordinate in modo decrescente:

$$f \propto \frac{1}{r} \implies f \cdot r \approx C, \quad (3.5)$$

dove C è una costante caratteristica del corpus. Nella generalizzazione analitica proposta da Mandelbrot (*Zipf-Mandelbrot law*):

$$f(r) \propto \frac{1}{(r + \beta)^\alpha}, \quad (3.6)$$

con parametri tipicamente calibrati a $\alpha \approx 1$ e $\beta \approx 2.7$.

Principio del Minimo Sforzo di Zipf (*Principle of Least Effort*)

George Kingsley Zipf teorizzò che questa distribuzione iperbolica emerga dall'equilibrio dinamico tra due forze cognitive contrapposte:

1. **Sforzo del Parlante (*Speaker*):** mira a minimizzare il dispendio energetico impiegando un vocabolario ristretto, costituito da parole corte, frequentissime e polisemiche.
2. **Sforzo dell'Ascoltatore (*Hearer*):** predilige un lessico ampio, formato da parole distinte, rare e precise, così da azzerare l'ambiguità e il costo di decodifica semantica.

Il compromesso evolutivo tra queste due esigenze di economia cognitiva produce la curva iperbolica di Zipf.

Dalla formulazione di Zipf discendono due proprietà strutturali:

- Il numero di significati m di una parola è inversamente proporzionale alla radice della sua frequenza: $m \propto 1/\sqrt{f}$ (oppure $m \propto 1/f$).
- Esiste una stretta correlazione inversa tra frequenza e lunghezza dei vocaboli: i termini ad altissima frequenza sono quasi invariabilmente i più brevi.

3.2.3 Feature Rare e Stop Words

Dall'andamento della curva di Zipf emergono due estremi con implicazioni decisive per l'architettura dei motori di ricerca:

1. **Feature Rare ed *Hapax Legomena*:** termini che compaiono una sola volta all'interno dell'intera collezione (*hapax legomena*). Rappresentano tipicamente tra il 40% e il 50% della dimensione totale del vocabolario $|V|$. Una feature può risultare rara per due motivi divergenti:
 - È un vocabolo specialistico a bassissima frequenza ma ad **altissimo contenuto informativo e discriminante** (es. "*arachnocentric*").
 - È un refuso ortografico (*typo*) o un identificatore sintetico privo di generalità (es. stringhe alfanumeriche di URL o hash di file).

La rimozione indiscriminata delle feature rare riduce drasticamente lo spazio occupato dall'indice, ma espone al rischio di eliminare chiavi di ricerca altamente specifiche.

2. **Stop Words:** parole ad altissima frequenza con ruolo prevalentemente sintattico-funzionale (“the”, “of”, “a”, “to”, “in”). Tradizionalmente, le stop words vengono filtrate nella fase di indicizzazione per risparmiare memoria e velocizzare l’elaborazione dei posting list (“the president of the united states of america” → “president united states america”).

Avvertenza sulle Liste di Stop Words

Le stop word list non hanno validità universale e dipendono dal dominio applicativo. Ad esempio, la lista predefinita dello stop word filter di MySQL includeva originariamente parole come “*appreciate*”, “*serious*” e “*unfortunately*”: termini che risulterebbero fatali se rimossi da un classificatore di *Sentiment Analysis* o da query mirate a recensioni e opinioni.

3.3 Acquisizione Dati e Parsing da Web: `urllib` e BeautifulSoup

Il punto di ingresso per la costruzione di una collezione documentale è il processo di crawling ed estrazione del contenuto testuale, schematizzato nella Figura 3.1.

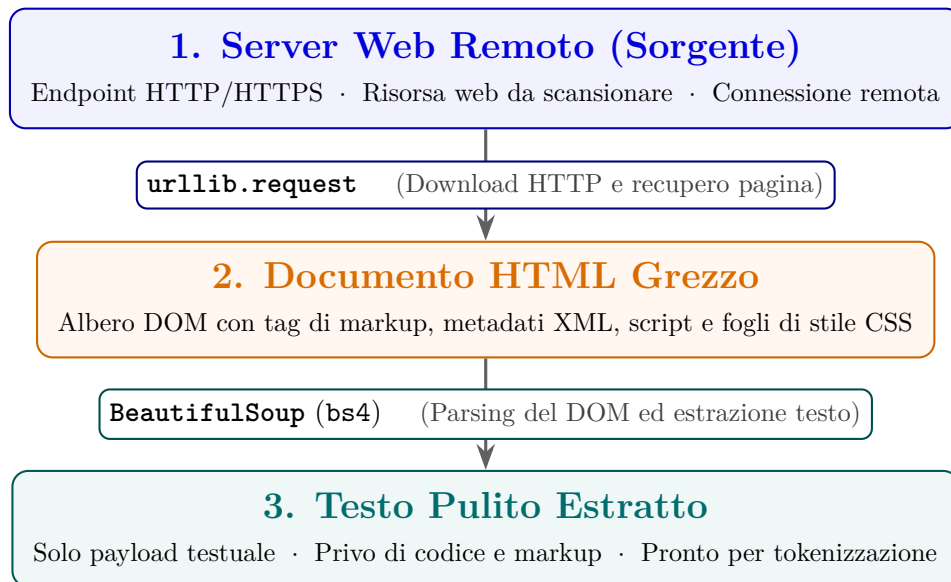


Figura 3.1: Pipeline di recupero HTTP e pulizia da markup del documento web.

3.3.1 Acquisizione con urllib

In Python, il protocollo HTTP/HTTPS viene gestito a basso livello tramite `urllib.request`:

Listing 3.1: Scaricamento grezzo di una risorsa web via HTTP.

```
1 from urllib import request
2
3 url = "http://hpc.isti.cnr.it/~nardini/"
4 response = request.urlopen(url)
5 html = response.read().decode('utf8')
6 # Restituisce l'intero albero HTML comprensivo di tag
   strutturali
```

3.3.2 Parsing e Normalizzazione con BeautifulSoup

I documenti web includono tag di rendering e script che costituirebbero rumore per il calcolo delle frequenze. La libreria BeautifulSoup analizza il DOM HTML ed estrae selettivamente il testo:

Listing 3.2: Estrazione del testo privo di markup con BeautifulSoup.

```
1 from bs4 import BeautifulSoup
2
3 soup = BeautifulSoup(html, "html5lib")
4
5 # Estrazione diretta dell'intero payload testuale
   concatenato:
6 text_all = soup.get_text()
7
8 # Estrazione granulare confinata ai soli paragrafi di
   testo (<p>):
9 text_paragraphs = [''.join(s.findAll(text=True)) for s
   in soup.findAll('p')]
```

3.4 Pipeline di Pre-elaborazione: Tokenizzazione e Sentence Splitting

Regola Empirica dell'Ingegneria dei Dati

Nei sistemi industriali di IR e NLP, l'**80% del tempo e dello sforzo ingegneristico** viene assorbito dalla pipeline di acquisizione, pulizia, normalizzazione e pre-elaborazione del testo prima di poter alimentare gli algoritmi di indicizzazione o machine learning.

3.4.1 Tokenizzazione

La tokenizzazione consiste nella segmentazione del testo grezzo in unità atomiche lessicali denominate **token**:

Listing 3.3: Tokenizzazione ed eliminazione di stop words con NLTK.

```
1 import nltk
2 from nltk import word_tokenize
3 from nltk.corpus import stopwords
4
5 text = "the president of the united states of america"
6 tokens = word_tokenize(text)
7 # Output: ['the', 'president', 'of', 'the', 'united', 'states', 'of', 'america']
8
9 stop_words = set(stopwords.words('english'))
10 features = set(tokens)
11 filtered_features = features.difference(stop_words)
12 # Output: {'america', 'president', 'states', 'united'}
```

3.4.2 Sentence Splitting e le Ambiguità della Punteggiatura

La suddivisione di un testo continuo in frasi indipendenti (*sentence boundary disambiguation*) è resa complessa dalla molteplicità d'uso del punto ortografico (.):

1. Delimitatore canonico di chiusura proposizionale.
2. Indicatore di acronimo, sigla o titolo onorifico (*U.S.A.*, *e.g.*, *St. Louis*, *Dr.*).

3. Separatore numerico o orario (*8.30*).
4. Sospensione discorsiva o ellissi (...).

Esempio 3.4.1 (Disambiguazione Sintattica delle Frasi). Si consideri il seguente enunciato discusso in aula:

“I’ll take the 8.30 train to St. Louis... or maybe I’ll stay in LA. Time will tell!”

Un parser ingenuo basato su `split(' ')` genererebbe errori madornali:

- Il punto in *8.30* è parte integrante di un valore temporale numerico.
- Il punto in *St.* appartiene all’abbreviazione per *Saint*.
- I punti di sospensione (...) rappresentano un’esitazione sintattica, senza chiudere la proposizione logica principale.
- Solo il punto successivo a *LA.* e il punto esclamativo in *Time will tell!* fungono da autentici terminatori di frase.

In NLTK, la funzione `sent_tokenize` adotta l’algoritmo non supervisionato *Punkt*, addestrato a riconoscere collocazioni lessicali e abbreviazioni per preservare i confini enunciativi corretti.

3.5 Costruzione del Vocabolario e Algoritmo di Conteggio

3.5.1 Definizione Formale di Vocabolario

Definizione 3.5.1 (Vocabolario della Collezione). Dato un corpus documentale $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$, il **Vocabolario** (o *Feature Set*, denotato con V o F) è l’unione dei tipi lessicali (*types*) distinti estratti da tutti i documenti:

$$V = \bigcup_{d \in \mathcal{D}} \{t \in d\}. \quad (3.7)$$

La cardinalità $|V|$ definisce la dimensionalità dello spazio vettoriale delle feature. È una grandezza strettamente **dataset-dependent**: qualsiasi inserimento o rimozione di documenti modifica il perimetro e la dimensione di V .

3.5.2 Discussione Algoritmica: Struttura Dati per il Conteggio

Durante la progettazione dell'algoritmo di conteggio delle frequenze di collezione (cf_t), è frequente l'errore di proporre la memorizzazione dei token in una struttura a insieme (**set**):

- **Inadeguatezza del set:** un insieme mantiene per definizione unicamente l'univocità delle chiavi eliminando qualsiasi duplicato al primo inserimento, impedendo così il tracciamento della frequenza numerica del termine.
- **Soluzione corretta:** una tabella hash (in Python, **dict** o **Counter**), avente il token lessicale come chiave e il conteggio cumulativo intero come valore associato.

L'Algoritmo 1 formalizza il processo di estrazione e computazione delle frequenze di collezione.

Algorithm 1: Costruzione del Vocabolario e Conteggio delle Frequenze di Collezione

Dati: Collezione di documenti $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$
Risultato: Vocabolario V e tabella delle frequenze freq_dict

```
1  $\text{freq\_dict} \leftarrow$  dizionario vuoto
2 foreach  $d \in \mathcal{D}$  do
3    $\text{tokens} \leftarrow \text{WordTokenize}(d)$ 
4   foreach  $t \in \text{tokens}$  do
5     if  $t \in \text{freq\_dict}$  then
6        $\text{freq\_dict}[t] \leftarrow \text{freq\_dict}[t] + 1$ 
7     else
8        $\text{freq\_dict}[t] \leftarrow 1$ 
9  $V \leftarrow \text{chiavi}(\text{freq\_dict})$ 
10 return  $V, \text{freq\_dict}$ 
```

Al termine della procedura, $|V| = \text{len}(\text{freq_dict})$ e per ciascun termine $t \in V$ è nota la frequenza totale cf_t .

3.6 Normalizzazione Morfologica: Stemming vs. Lemmatizzazione

Per raggruppare varianti flesse della stessa parola sotto un'unica forma canonica, si adottano due metodologie contrapposte, confrontate nella Tabella 3.1.

Parametro	Stemming	Lemmatizzazione
Obiettivo	Ridurre la parola alla radice minima (<i>stem</i>) tramite euristiche.	Ridurre la parola al lemma canonico esistente a vocabolario.
Metodologia	Troncamento algoritmico basato su regole prefissate di riscrittura suffissi.	Analisi morfologica e morfosintattica completa basata su dizionario e grafi.
Correttezza lessicale	Bassa: genera frequentemente non-parole o radici inesistenti.	Alta: produce sempre forme canoniche lessicalmente e grammaticalmente valide.
Dipendenza da POS	Nessuna: opera puramente sulla stringa di caratteri.	Fondamentale: necessita del <i>Part-of-Speech tagging</i> contestuale.
Costo computazionale	Minimo: trasformazioni su stringhe ultra-rapide in tempo costante.	Elevato: richiede inferenza contestuale su modelli o interrogazione a database lessicali.

Tabella 3.1: Confronto sistematico tra Stemming euristico e Lemmatizzazione morfosintattica.

3.6.1 Stemming ed Esempio Porter Stemmer

Lo stemmer applica regole sequenziali di eliminazione o sostituzione di suffissi tipici (es. -ing, -s, -ed).

Listing 3.4: Stemming con PorterStemmer ed evidenza dell'errore lessicale.

```

1 from nltk.stem.porter import PorterStemmer
2
3 stemmer = PorterStemmer()
4 stemmer.stem('cars') # Risultato: 'car' (corretto)
5 stemmer.stem('was')  # Risultato: 'wa' (ERRORE
                        LESSICALE)

```

Origine dell'Errore dello Stemmer su “was”

Lo stemmer individua la lettera finale 's' in “was” e applica ciecamente la regola grammaticale di riduzione del plurale inglese, rimuovendo la desinenza. Non disponendo di alcuna informazione sulla categoria sintattica (voce del verbo *to be*), produce la pseudo-radice inesistente 'wa'.

3.6.2 Lemmatizzazione con WordNet

La lemmatizzazione consulta un dizionario lessicale strutturato (come WordNet). Se non viene fornito il tag POS, l'algoritmo assume come ipotesi predefinita che il termine sia un sostantivo:

Listing 3.5: Lemmatizzazione contestuale con WordNetLemmatizer.

```
1 from nltk.stem.wordnet import WordNetLemmatizer
2
3 lmtzr = WordNetLemmatizer()
4 lmtzr.lemmatize('cars')           # Output: 'car'
5 lmtzr.lemmatize('was')           # Output: 'wa'
   (default: sostantivo)
6 lmtzr.lemmatize('was', pos='v')   # Output: 'be'
   (FORMA CORRETTA: verbo essere)
```

3.6.3 Il Trade-off Ingegneristico nei Motori di Ricerca

Nei motori di ricerca commerciali su scala planetaria, lo **Stemming** è rimasto storicamente la soluzione dominante per l'indicizzazione: pur introducendo distorsioni ortografiche occasionali, richiede un costo computazionale infinitesimo rispetto all'esecuzione di un POS tagger e di una disambiguazione su miliardi di pagine web.

3.7 Il Modello Bag of Words (BoW)

3.7.1 Transizione da Sequenza a Insieme

Un testo non strutturato nasce originariamente come sequenza ordinata di token:

$$\text{text} \longrightarrow [t_1, t_2, t_3, \dots, t_k]. \quad (3.8)$$

Questa rappresentazione sequenziale pone due complessità: lunghezza variabile tra testi differenti e difficoltà di mappatura su uno spazio a dimensionalità fissa.

Il modello **Bag of Words (BoW)** astrae il documento collassando la sequenza dei token in un insieme di termini unici (o multinsieme con frequenze):

Listing 3.6: Rappresentazione insiemistica Bag of Words.

```
1 text = 'the president of the united states of america'
2 tokens = word_tokenize(text)          # Lista di 8 elementi
3 bow = set(tokens)                     # Insieme di 6
    elementi unici
4 # bow: {'america', 'of', 'president', 'states', 'the', '
    united'}
```

3.7.2 Limiti Strutturali del BoW

1. **Perdita della frequenza locale:** nella variante insiemistica pura si annulla il numero di volte in cui una parola ricorre nel testo.
2. **Perdita totale dell'ordine lessicale e della sintassi:** il BoW assume che la posizione reciproca delle parole sia ininfluyente per la determinazione del significato del documento.

Esempio 3.7.1 (Collasso Semantico nel Modello Bag of Words). Si considerino due proposizioni aventi significato diametralmente opposto:

- t_1 : “*I won, and thus you lose.*” (celebra la vittoria dell'emittente)
- t_2 : “*I lose, and thus you won.*” (ammette la sconfitta dell'emittente)

Eseguendo la rappresentazione BoW insiemistica in Python:

Listing 3.7: Dimostrazione dell'indistinguibilità semantica in BoW.

```
1 t1 = set(word_tokenize('I won, and thus you lose.'))
2 t2 = set(word_tokenize('I lose, and thus you won.'))
3
4 # Insieme t1: {'and', 'lose', 'thus', 'won', 'you'}
5 # Insieme t2: {'and', 'lose', 'thus', 'won', 'you'}
6 print(t1 == t2) # Output: True
```

Per la macchina, sotto l'ipotesi BoW, i due enunciati sono identici. Questo limite nasce dal fatto che la semantica umana è **composizionale**, dipendendo in modo vincolante dall'ordine sintattico dei costituenti.

3.8 N-grammi: Word n-grams e Character n-grams

Per ovviare alla perdita dell'ordine senza dover ricorrere a costosi alberi di parsing sintattico, si estende il vocabolario tramite gli **n-grammi**.

3.8.1 Word n-grams

Un **word n-gram** è una sequenza contigua di n parole estratte dal testo. Preserva le relazioni di adiacenza e l'ordine locale:

Listing 3.8: Distinzione semantica locale mediante bigrammi di parole.

```
1 import nltk
2
3 t1 = set(nltk.ngrams(nltk.word_tokenize('I won, and thus
4     , you lose.'), 2))
5
6 t2 = set(nltk.ngrams(nltk.word_tokenize('I lose, and
7     thus, you won.'), 2))
8
9 # t1 genera: ('won', 'and'), ('you', 'lose'), ...
10 # t2 genera: ('lose', 'and'), ('you', 'won'), ...
11 print(t1 == t2) # Output: False
```

Tramite bigrammi ($n = 2$), la rappresentazione preserva il legame tra soggetto e predicato verbale, distinguendo correttamente i due enunciati.

3.8.2 Character n-grams

I **character n-grams** scompongono i termini in sottostringhe contigue di lunghezza n (tipicamente trigrammi di caratteri, $n = 3$). Questa tecnica è fondamentale per garantire la tolleranza ai refusi di battitura (*typo tolerance*) e catturare affinità morfologiche:

Listing 3.9: Resistenza ai refusi tramite trigrammi di caratteri.

```
1 t1 = set(nltk.ngrams('rainbow', 3))
2 t2 = set(nltk.ngrams('rainbaw', 3)) # Contiene un typo:
3     'a' al posto di 'o'
4
5 intersezione = t1.intersection(t2)
6 # Output: {('r', 'a', 'i'), ('a', 'i', 'n'), ('i', 'n',
7     'b')}
```

Nonostante l'errore ortografico sul singolo carattere, i due vocaboli condividono il 60% dei loro trigrammi (3 su 5), consentendo al motore di stimare una somiglianza metrica elevata e proporre correzioni (*“Forse cercavi...”*).

3.9 Il Modello a Spazio Vettoriale (Vector Space Model - VSM)

Nel **Vector Space Model** (VSM), il corpus documentale e le query dell'utente vengono mappati in uno spazio vettoriale reale a dimensionalità finita:

$$\text{Dimensione dello spazio} = |F| = |V|. \quad (3.9)$$

Ogni asse ortogonale dello spazio corrisponde a uno specifico termine $f \in V$. Sia i documenti d sia le query q sono rappresentati come vettori immersi in tale spazio:

$$\vec{v}(d) \in \mathbb{R}^{|V|}, \quad \vec{v}(q) \in \mathbb{R}^{|V|}. \quad (3.10)$$

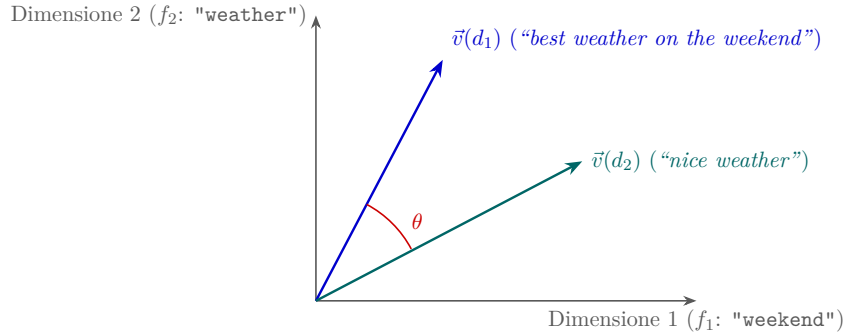


Figura 3.2: Rappresentazione geometrica bidimensionale di documenti nel Vector Space Model.

3.9.1 Mappatura Formale tramite Vettori Canonici Unitari

Ciascuna feature $f_i \in V$ corrisponde a un vettore base canonico one-hot dello spazio reale:

$$\vec{v}(f_1) = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \quad \vec{v}(f_2) = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \quad \vec{v}(f_{|V|}) = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}. \quad (3.11)$$

Un generico documento d viene formalizzato come la **combinazione lineare pesata** dei vettori base dei termini che vi compaiono:

$$\vec{v}(d) = \sum_{f \in d} w_{f,d} \vec{v}(f), \quad (3.12)$$

dove $w_{f,d} \in \mathbb{R}$ indica il peso (l'importanza informativa) del termine f nel documento d .

3.9.2 Sparsità Estrema dei Vettori di Documento

Si consideri un vocabolario di taglia reale $|V| = 10^6$ (un milione di termini) e un documento medio d composto da 200 vocaboli unici:

$$\vec{v}(d) = [0, w_{t_1,d}, 0, \dots, 0, w_{t_{200},d}, 0, \dots, 0]^T. \quad (3.13)$$

La frazione di componenti non nulle del vettore è:

$$\frac{|\{i \mid v_i(d) \neq 0\}|}{|V|} = \frac{200}{1\,000\,000} = 0.0002 = 0.02\%. \quad (3.14)$$

Oltre il **99.98%** dei valori nel vettore è rigorosamente pari a zero. Rappresentare tali vettori come array densi in virgola mobile a 32 bit (4 MB per singolo documento) comporterebbe un collasso della memoria di sistema. Risulta pertanto inderogabile l'adozione di strutture per dati sparsi (*sparse vectors*, mappe chiave-valore o l'*Inverted Index*).

Capitolo 4

Modelli di Retrieval e Ranking: Spazio Vettoriale e BM25

In questo capitolo vengono formalizzati i principali modelli di Information Retrieval dedicati all'assegnazione di un punteggio di rilevanza continuo (*ranking*) per graduare i documenti rispetto a una query utente. Partendo dall'analisi critica e dai limiti intrinseci del modello booleano esatto, si esamina il coefficiente di similarità di Jaccard, si approfondiscono i fondamenti della ponderazione statistica TF-IDF e della notazione SMART nello Spazio Vettoriale (VSM), per poi analizzare nel dettaglio la funzione di scoring probabilistica Okapi BM25 e la sintesi architetturale dell'intera pipeline di recupero.

4.1 Modelli di Retrieval: Da Booleano a Ranked Retrieval

Proprietà	Retrieval Booleano	Ranked Retrieval
Criterio di selezione	Corrispondenza logica esatta (dentro/fuori).	Punteggio di similarità continuo $\text{Score}(q, d) \in \mathbb{R}$.
Sintassi query	Espressioni logiche vincolanti (AND, OR, NOT).	Linguaggio naturale libero (<i>free text query</i>).
Output fornito	Insieme non ordinato di documenti corrispondenti.	Graduatoria ordinata dei migliori <i>Top-k</i> risultati.
Criticità utente	Problema di <i>Feast or Famine</i> .	Nessun sovraccarico cognitivo: restituzione calibrata.

Tabella 4.1: Confronto concettuale tra Retrieval Booleano esatto e Ranked Retrieval graduato.

4.1.1 La Patologia del Retrieval Booleano: “Feast or Famine”

Nel modello booleano esatto, l’utente comune incontra gravi difficoltà nel dosare gli operatori logici:

- **Famine (Carestia):** se la query è eccessivamente restrittiva (vincoli AND), nessun documento soddisfa tutte le clausole, producendo zero risultati.
- **Feast (Abbondanza incontrollata):** se la query è generica o fa uso di operatori OR, il sistema restituisce decine di migliaia di documenti indifferenziati.

Nel **Ranked Retrieval**, il sistema calcola per ogni coppia formato da query e documento un punteggio di rilevanza continua:

$$\text{Score}(q, d) \in \mathbb{R}. \quad (4.1)$$

I documenti vengono quindi ordinati per punteggio decrescente e presentati all’utente in forma di graduatoria, mostrando i primi k risultati più rilevanti.

4.2 Coefficiente di Jaccard e sue Limitazioni nello Scoring

Un primo tentativo intuitivo per assegnare un punteggio continuo a una coppia query-documento consiste nel considerare query e documento come insiemi di parole A e B , applicando il coefficiente di similarità di Jaccard.

4.2.1 Definizione Formale

Definizione 4.2.1 (Coefficiente di Jaccard). Dati due insiemi finiti di termini A e B :

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}. \quad (4.2)$$

Gode delle seguenti proprietà:

- $J(A, A) = 1$,
- $J(A, B) = 0 \iff A \cap B = \emptyset$,
- $0 \leq J(A, B) \leq 1$.

4.2.2 Esempio Guidato e Fallimento del Punteggio di Jaccard

Esempio 4.2.1 (Il Paradosso di Jaccard su Query e Documenti Reali). Si consideri la query q e due documenti candidati:

- **Query q :** “*ides of march*” con $A = \{\text{ides, of, march}\}$, $|A| = 3$.
- **Doc 1 (d_1):** “*caesar died in march*” con $B_1 = \{\text{caesar, died, in, march}\}$, $|B_1| = 4$.
- **Doc 2 (d_2):** “*the long march*” con $B_2 = \{\text{the, long, march}\}$, $|B_2| = 3$.

Calcolando i coefficienti di similarità insiemistica di Jaccard:

$$\begin{aligned} |A \cap B_1| = 1, \quad |A \cup B_1| = 6 &\implies J(q, d_1) = \frac{1}{6} \approx 0.166; \\ |A \cap B_2| = 1, \quad |A \cup B_2| = 5 &\implies J(q, d_2) = \frac{1}{5} = 0.200. \end{aligned}$$

Risultato paradossale: Il sistema giudica d_2 (“*the long march*”, la Lunga Marcia di Mao Tse-tung) più rilevante per la query sulle Idi di Marzo rispetto a d_1 (“*caesar died in march*”, la morte di Giulio Cesare). Questa distorsione avviene unicamente perché d_2 contiene una parola in meno di d_1 , a parità di un unico termine sovrapposto (“*march*”).

4.2.3 Perché Jaccard è Inadeguato per l'Information Retrieval

Il coefficiente di Jaccard fallisce come funzione di ranking per tre motivi teorici e pratici:

1. **Ignora la Term Frequency (tf):** tratta i documenti come insiemi booleani di parole distinte; un termine ripetuto 10 volte in un passaggio pesa esattamente quanto un termine che compare una volta fugace.
2. **Ignora l'Informatività Globale dei Termini (nessun idf):** non distingue vocaboli rari e discriminanti da stop word frequentissime. La sovrapposizione sulla preposizione “*of*” incrementa il coefficiente esattamente quanto la sovrapposizione su “*ides*”.
3. **Penalizzazione impropria della lunghezza documentale:** la cardinalità dell'unione al denominatore $|A \cup B|$ cresce rapidamente con la taglia del documento, penalizzando severamente e ingiustamente i documenti estesi rispetto ai testi stringati.

4.3 Matrici Termine–Documento e Ponderazione TF-IDF

4.3.1 Dalle Matrici di Incidenza Binaria alle Matrici di Conteggio

Nelle formulazioni booleane, una collezione è descritta da una matrice termine-documento binaria $M \in \{0, 1\}^{|V| \times |D|}$. Nel modello vettoriale pesato, gli elementi della matrice rappresentano la frequenza locale (*Term Frequency*) del termine t nel documento d :

$$C_{t,d} = \text{tf}_{t,d} \in \mathbb{N}. \quad (4.3)$$

Dalla Matrice Teorica alle Strutture Dati Fisiche

La matrice termine–documento costituisce una potente astrazione teorica, ma la sua memorizzazione diretta in forma densa è impraticabile a causa della sparsità estrema ($> 99.99\%$). La trasformazione di questa matrice in strutture dati ad alte prestazioni (*Forward Index* e *Index Invertito*) e le relative tecniche di compressione per interi vengono formalizzate nel Capitolo 5.

4.3.2 La Term Frequency Logaritmica

L'utilità informativa di un termine non cresce linearmente con il numero di volte in cui viene ripetuto. Un documento in cui la parola “*computer*” ricorre 10 volte è sicuramente più rilevante di uno in cui compare 1 volta, ma **non dieci volte più rilevante**. Per modellare questo rendimento marginale decrescente, si applica lo smorzamento logaritmico:

$$w_{t,d} = \begin{cases} 1 + \log_{10}(\text{tf}_{t,d}) & \text{se } \text{tf}_{t,d} > 0, \\ 0 & \text{se } \text{tf}_{t,d} = 0. \end{cases} \quad (4.4)$$

La Tabella 4.2 riporta la compressione dinamica della frequenza operata dalla formula:

$\text{tf}_{t,d}$ grezzo	$w_{t,d} = 1 + \log_{10}(\text{tf})$	Interpretazione
0	0.000	Termine assente nel documento
1	1.000	Singola comparsa
2	≈ 1.301	Incremento moderato
10	2.000	Raddoppio del peso per frequenza decuplicata
1 000	4.000	Peso quadruplicato per mille occorrenze

Tabella 4.2: Scala di compressione del peso tramite Term Frequency logaritmica.

Lo score preliminare di similarità tra query q e documento d basato sulla sola frequenza logaritmica è:

$$\text{Score}_{\text{tf}}(q, d) = \sum_{t \in q \cap d} (1 + \log_{10}(\text{tf}_{t,d})). \quad (4.5)$$

4.3.3 Document Frequency (df_t) vs. Collection Frequency (cf_t)

Per quantificare la specificità globale di un vocabolo nell'intero archivio documentale $\mathcal{D}$, occorre distinguere due metriche:

- **Collection Frequency (cf_t):** il numero totale di occorrenze di t sommato su tutti i documenti della collezione.
- **Document Frequency (df_t):** il numero di documenti distinti in cui il termine t compare almeno una volta ($df_t \leq N$, con $N = |\mathcal{D}|$).

Esempio 4.3.1 (Confronto Didattico tra cf_t e df_t). Si consideri l'analisi empirica condotta su un corpus di grandi dimensioni:

- Termine “*Insurance*”: $cf = 10\,440$, $df = 3\,997$.
- Termine “*Try*”: $cf = 10\,422$, $df = 8\,760$.

Benché le frequenze aggregate di collezione siano pressoché identiche ($cf \approx 10\,430$), il vocabolo “*Insurance*” si concentra in meno della metà dei documenti rispetto a “*Try*”. Di conseguenza, “*Insurance*” costituisce un indicatore tematico straordinariamente più discriminante e specifico, dovendo ricevere un peso selettivo molto maggiore.

4.3.4 La Formulazione dell’Inverse Document Frequency (IDF)

Per assegnare un peso elevato ai termini rari e penalizzare i termini ubiqui distribuiti sull’intero archivio, si formalizza l’**Inverse Document Frequency**:

$$\text{idf}_t = \log_{10} \left(\frac{N}{df_t} \right). \quad (4.6)$$

La funzione logaritmica attenua l’ampiezza dinamica del rapporto N/df_t .

La Tabella 4.3 illustra i valori di idf_t calcolati su una collezione di $N = 1\,000\,000$ documenti:

Termine t	df_t	N/df_t	$\text{idf}_t = \log_{10}(N/df_t)$
calpurnia	1	1 000 000	6.0
animal	100	10 000	4.0
sunday	1 000	1 000	3.0
fly	10 000	100	2.0
under	100 000	10	1.0
the	1 000 000	1	0.0

Tabella 4.3: Valori di IDF per termini a frequenza documentale crescente ($N = 1\,000\,000$).

Indipendenza dell’IDF dalla Query

I pesi idf_t dipendono esclusivamente dalla distribuzione statistica della collezione documentale e sono **completamente indipendenti dalla query**. Possono pertanto essere precalcolati e indicizzati offline durante la costruzione dell’indice invertito.

4.3.5 Impatto dell'IDF sulla Graduatoria

- **Nelle query monoterminare (es. “iPhone”):** l'IDF non modifica in alcun modo l'ordinamento relativo dei documenti. Essendo identico per tutti i documenti contenenti il termine, funge da costante moltiplicativa uniforme che scala gli score senza alterare le posizioni relative:

$$\text{Score}(q, d) = (1 + \log_{10} \text{tf}_{t,d}) \cdot \text{idf}_t. \quad (4.7)$$

- **Nelle query multiterminare (es. “capricious person”):** l'IDF assume un ruolo determinante; il termine specialistico raro (“capricious”, basso df , alto idf) domina lo score complessivo rispetto al termine comune generico (“person”, alto df , basso idf).

4.3.6 Lo Schema di Ponderazione TF-IDF Globale

Moltiplicando il peso locale nel documento per la specificità globale, si ottiene la classica formula del peso **TF-IDF**:

$$w_{t,d} = \text{tf-idf}_{t,d} = (1 + \log_{10} \text{tf}_{t,d}) \cdot \log_{10} \left(\frac{N}{df_t} \right). \quad (4.8)$$

4.4 Similarità Coseno e Normalizzazione della Lunghezza

4.4.1 Perché la Distanza Euclidea Fallisce per i Documenti

Dati i vettori pesati di due documenti $\vec{x}$ e $\vec{y}$ in $\mathbb{R}^{|V|}$, la distanza euclidea classica è:

$$d_E(\vec{x}, \vec{y}) = \|\vec{x} - \vec{y}\|_2 = \sqrt{\sum_{i=1}^{|V|} (x_i - y_i)^2}. \quad (4.9)$$

Esempio 4.4.1 (Esperimento Ideale del Documento Concatenato). Si consideri un documento breve d . Si generi un nuovo documento d' concatenando due copie consecutive di d ($d' = d \circ d$):

- Sotto il profilo semantico e tematico, d e d' trattano rigorosamente lo stesso argomento con le medesime proporzioni lessicali.

- Tuttavia, ciascuna frequenza $\text{tf}_{t,d'}$ raddoppia; di conseguenza il vettore $\vec{v}(d')$ possiede componenti pari al doppio di $\vec{v}(d)$, raddoppiandone la norma geometrica: $\|\vec{v}(d')\|_2 \approx 2\|\vec{v}(d)\|_2$.
- La distanza euclidea $d_E(\vec{v}(d), \vec{v}(d'))$ risulterà considerevole, inducendo il sistema a classificare come dissimili due testi semanticamente indistinguibili.

4.4.2 L'Angolo come Misura di Similarità e il Coseno

Poiché i vettori $\vec{v}(d)$ e $\vec{v}(d')$ giacciono sul medesimo raggio vettore nello spazio geometrico, l'angolo θ sotteso tra loro è pari a 0° . L'angolo misura la concordanza tematica prescindendo dalla lunghezza del testo.

Poiché nell'intervallo $[0^\circ, 180^\circ]$ la funzione trigonometrica del coseno è strettamente decrescente:

$$\text{Minimizzare l'angolo } \theta \iff \text{Massimizzare } \cos(\theta). \quad (4.10)$$

Si adotta pertanto la **Similarità Coseno**:

$$\cos(\theta) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\|_2 \|\vec{d}\|_2} = \frac{\sum_{i=1}^{|V|} q_i d_i}{\sqrt{\sum_{i=1}^{|V|} q_i^2} \sqrt{\sum_{i=1}^{|V|} d_i^2}}. \quad (4.11)$$

4.4.3 Normalizzazione Euclidea L_2

Dividendo un vettore per la sua norma euclidea $\|\vec{x}\|_2$, lo si proietta sulla superficie dell'ipersfera unitaria ($\|\vec{x}'\|_2 = 1$). Quando tutti i vettori sono preventivamente normalizzati in lunghezza, la similarità coseno si riduce al semplice **prodotto scalare**:

$$\cos(\vec{q}, \vec{d}) = \vec{q} \cdot \vec{d} = \sum_{i=1}^{|V|} q_i d_i \quad \text{per } \|\vec{q}\|_2 = \|\vec{d}\|_2 = 1. \quad (4.12)$$

4.4.4 Esercizio Guidato Completo: Similarità tra Tre Romanzi Classici

Si confrontano tre opere letterarie:

- **SaS:** *Sense and Sensibility* (Jane Austen)
- **PaP:** *Pride and Prejudice* (Jane Austen)
- **WH:** *Wuthering Heights* (Emily Brontë)

Termine	SaS	PaP	WH
affection	115	58	20
jealous	10	7	11
gossip	2	0	6
wuthering	0	0	38

Tabella 4.4: Frequenze grezze nei tre romanzi su un sottoinsieme di termini.

Passo 0: Frequenze Grezze (Term Counts).

Passo 1: Pesi Logaritmici ($w = 1 + \log_{10}(\text{tf})$ per $\text{tf} > 0$, altrimenti 0).

- **SaS:** $w(\text{affection}) \approx 3.06$, $w(\text{jealous}) = 2.00$, $w(\text{gossip}) \approx 1.30$, $w(\text{wuthering}) = 0.00$.
- **PaP:** $w(\text{affection}) \approx 2.76$, $w(\text{jealous}) \approx 1.85$, $w(\text{gossip}) = 0.00$, $w(\text{wuthering}) = 0.00$.
- **WH:** $w(\text{affection}) \approx 2.30$, $w(\text{jealous}) \approx 2.04$, $w(\text{gossip}) \approx 1.78$, $w(\text{wuthering}) \approx 2.58$.

Passo 2: Calcolo delle Norme Euclidee L_2 e Normalizzazione.

$$\begin{aligned}\|\vec{v}(\text{SaS})\|_2 &= \sqrt{3.06^2 + 2.00^2 + 1.30^2} = \sqrt{15.0536} \approx 3.88, \\ \|\vec{v}(\text{PaP})\|_2 &= \sqrt{2.76^2 + 1.85^2} = \sqrt{11.0401} \approx 3.32, \\ \|\vec{v}(\text{WH})\|_2 &= \sqrt{2.30^2 + 2.04^2 + 1.78^2 + 2.58^2} = \sqrt{19.2764} \approx 4.39.\end{aligned}$$

Dividendo ciascuna componente per la propria norma si ottengono i vettori unitari normalizzati:

- $\vec{v}'(\text{SaS}) \approx [0.789, 0.515, 0.335, 0.000]^T$,
- $\vec{v}'(\text{PaP}) \approx [0.832, 0.555, 0.000, 0.000]^T$,
- $\vec{v}'(\text{WH}) \approx [0.524, 0.465, 0.405, 0.588]^T$.

Passo 3: Calcolo delle Similarità Coseno (Prodotti Scalari).

$$\begin{aligned}\cos(\text{SaS}, \text{PaP}) &\approx (0.789 \cdot 0.832) + (0.515 \cdot 0.555) \approx \mathbf{0.94}, \\ \cos(\text{SaS}, \text{WH}) &\approx (0.789 \cdot 0.524) + (0.515 \cdot 0.465) + (0.335 \cdot 0.405) \approx \mathbf{0.79}, \\ \cos(\text{PaP}, \text{WH}) &\approx (0.832 \cdot 0.524) + (0.555 \cdot 0.465) \approx \mathbf{0.69}.\end{aligned}$$

Interpretazione Empirica

La similarità coseno individua con precisione la stretta affinità tematica e stilistica tra le due opere della stessa autrice Jane Austen:

$$\cos(\text{SaS}, \text{PaP}) > \cos(\text{SaS}, \text{WH}) > \cos(\text{PaP}, \text{WH}), \quad (4.13)$$

ossia $0.94 > 0.79 > 0.69$, distinguendo nettamente i romanzi di Austen dal capolavoro gotico di Emily Brontë.

4.5 Notazione SMART e Varianti di TF-IDF

I motori di ricerca consentono di combinare formule differenti di TF, IDF e normalizzazione per i documenti rispetto alle query. Tale schema combinatorio è standardizzato tramite la **notazione SMART** nella forma:

$$\text{ddd.qqq}, \quad (4.14)$$

dove la prima terna codifica i pesi del documento (d) e la seconda quelli della query (q).

4.5.1 Tabella delle Convenzioni SMART

Term Frequency (TF)	Document Frequency (IDF)	Normalizzazione
n $\text{tf}_{t,d}$ (naturale)	n 1 (nessun IDF)	n 1 (nessuna)
l $1 + \log_{10}(\text{tf}_{t,d})$	t $\log_{10}(N/df_t)$	c $\frac{1}{\sqrt{w_1^2 + \dots + w_M^2}}$ (L_2)
a $0.5 + \frac{0.5 \cdot \text{tf}}{\max_t(\text{tf})}$	p $\max\left(0, \log_{10} \frac{N - df_t}{df_t}\right)$	u Pivoted unique length
b 1 se $\text{tf} > 0$, altrimenti 0		b Byte size: $1/\text{len}^\alpha$
L $\frac{1 + \log(\text{tf})}{1 + \log(\text{avg_tf})}$		

Tabella 4.5: Convenzione SMART per la composizione modulare dei pesi vettoriali.

4.5.2 Lo Schema Standard **lnc.1tc**

Uno degli schemi SMART più collaudati ed efficienti in IR è **lnc.1tc**:

- **Documento (lnc)**: log-frequency (l), nessun IDF (n), normalizzazione coseno L_2 (c).
- **Query (1tc)**: log-frequency (1), standard IDF (t), normalizzazione coseno L_2 (c).

4.5.3 Esempio Calcolato Completo: Schema Inc.ltc

Si consideri il documento d (“*car insurance auto insurance*”) e la query q (“*best car insurance*”) su una collezione con $N = 1\,000\,000$ documenti:

$$\begin{aligned} df(\text{auto}) &= 5\,000 \implies idf = \log_{10}(200) \approx 2.30, \\ df(\text{best}) &= 50\,000 \implies idf = \log_{10}(20) \approx 1.30, \\ df(\text{car}) &= 10\,000 \implies idf = \log_{10}(100) = 2.00, \\ df(\text{insurance}) &= 1\,000 \implies idf = \log_{10}(1000) = 3.00. \end{aligned}$$

Termine	Query (q): schema ltc					Documento (d): schema Inc				Prodotto $\vec{q}' \cdot \vec{d}'$
	tf _{raw}	tf _{wt}	idf	wt = tf · idf	n'lize	tf _{raw}	tf _{wt}	wt	n'lize	
auto	0	0	2.30	0.00	0.00	1	1.00	1.00	0.52	0.0000
best	1	1	1.30	1.30	0.34	0	0.00	0.00	0.00	0.0000
car	1	1	2.00	2.00	0.52	1	1.00	1.00	0.52	0.2704
insurance	1	1	3.00	3.00	0.78	2	1.30	1.30	0.68	0.5304

Tabella 4.6: Calcolo passo-passo del punteggio di similarità con schema SMART Inc.ltc.

Verifica della Normalizzazione dei Vettori:

$$\begin{aligned} \|\vec{q}'\|_2 &= \sqrt{1.30^2 + 2.00^2 + 3.00^2} = \sqrt{14.69} \approx 3.83, \\ q'_{\text{best}} &\approx 0.34, \quad q'_{\text{car}} \approx 0.52, \quad q'_{\text{ins}} \approx 0.78; \\ \|\vec{d}'\|_2 &= \sqrt{1.00^2 + 1.00^2 + 1.30^2} = \sqrt{3.69} \approx 1.92, \\ d'_{\text{auto}} &\approx 0.52, \quad d'_{\text{car}} \approx 0.52, \quad d'_{\text{ins}} \approx 0.68. \end{aligned}$$

Calcolo dello Score Finale:

$$\begin{aligned} \text{Score}(q, d) &= \sum_t q'_t \cdot d'_t \\ &= (0.52 \cdot 0.52) + (0.78 \cdot 0.68) = 0.2704 + 0.5304 = \mathbf{0.80}. \end{aligned} \quad (4.15)$$

4.6 Best Match 25 (Okapi BM25)

Sviluppato negli anni '80 e '90 da Stephen Robertson e Karen Spärck Jones all'interno del sistema sperimentale Okapi presso la City University di Londra, **BM25** (*Best Match 25*) è una funzione di ranking non-lineare basata sul modello probabilistico di reperimento. Costituisce tuttora la baseline di riferimento più solida e performante per il reperimento lessicale primario (*first-stage retrieval*) nei motori di ricerca industriali.

4.6.1 Formulazione Matematica di BM25

Dato un documento D e una query a testo libero $Q = \{q_1, q_2, \dots, q_n\}$, il punteggio BM25 è:

$$\text{Score}_{\text{BM25}}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdL}}\right)}, \quad (4.16)$$

dove l'IDF probabilistico di Robertson-Spärck Jones è formulato come:

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right), \quad (4.17)$$

con le seguenti variabili operative:

- N : numero totale di documenti nell'intera collezione $\mathcal{D}$,
- $n(q_i)$: numero di documenti contenenti il termine di query q_i (equivalente a df_{q_i}),
- $f(q_i, D)$: frequenza grezza del termine q_i nel documento D (pari a $tf_{q_i, D}$),
- $|D|$: lunghezza complessiva del documento misurata in numero di token,
- avgdL : lunghezza media dei documenti nell'intera collezione (*average document length*).

4.6.2 Ruolo e Calibrazione degli Iperparametri

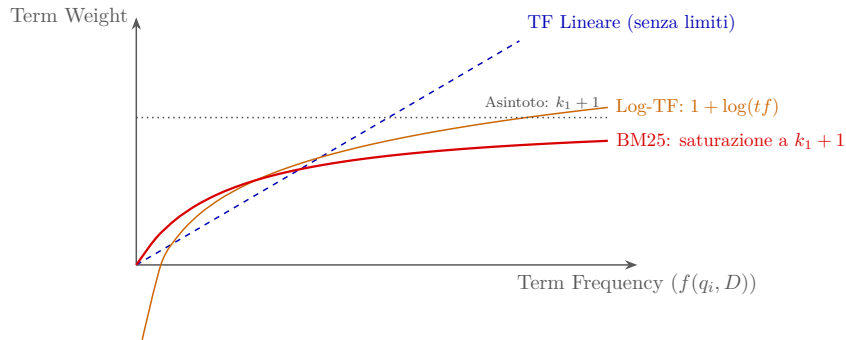


Figura 4.1: Confronto qualitativo tra crescita del TF lineare, TF logaritmico e saturazione asintotica in BM25.

1. **Parametro $k_1 \in [1.2, 2.0]$ (Saturazione della frequenza):** Controlla la rapidità con cui il contributo della Term Frequency converge asintoticamente al proprio limite superiore. Al tendere della frequenza locale all'infinito:

$$\lim_{f(q_i, D) \rightarrow \infty} \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + \dots} = k_1 + 1. \quad (4.18)$$

A differenza del modello TF-IDF classico in cui il peso continua a salire indefinitamente, in BM25 il contributo informativo di un singolo termine è rigorosamente limitato superiormente da $k_1 + 1$. *Caso limite:* se $k_1 = 0$, la componente di frequenza si annulla e il modello degenera in un ranking puramente binario basato unicamente su presenza/assenza e IDF.

2. **Parametro $b \in [0, 1]$ (Penalizzazione della lunghezza documentale):** Regola l'incidenza della lunghezza del documento rispetto alla media della collezione:

$$B = \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}} \right). \quad (4.19)$$

- Se $b = 1$, la normalizzazione è totale: un documento lungo il doppio rispetto alla media deve presentare il doppio delle occorrenze del termine per vedersi riconosciuto lo stesso score.
- Se $b = 0$, la normalizzazione è disattivata ($B = 1$ costante): la lunghezza del documento viene totalmente ignorata.
- Valore standard consigliato in letteratura: $b \approx 0.75$.

Valutazione di BM25 su Larga Scala

La formula di Okapi BM25 definisce la funzione analitica di scoring per ciascuna coppia query-documento. Tuttavia, nei moderni motori di ricerca con miliardi di documenti, la valutazione non può avvenire scansionando l'intera collezione. L'esecuzione ottimizzata di BM25 tramite liste invertite e gli algoritmi di query processing (*Term-at-a-Time* e *Document-at-a-Time* con tecniche di pruning dinamico come WAND) vengono trattati in dettaglio nel Capitolo 5.

4.7 Mappa Concettuale Riassuntiva

La Figura 4.2 offre una visione d'insieme del flusso completo di elaborazione: dall'acquisizione della risorsa web grezza fino alle funzioni di scoring vettoriale e probabilistico.

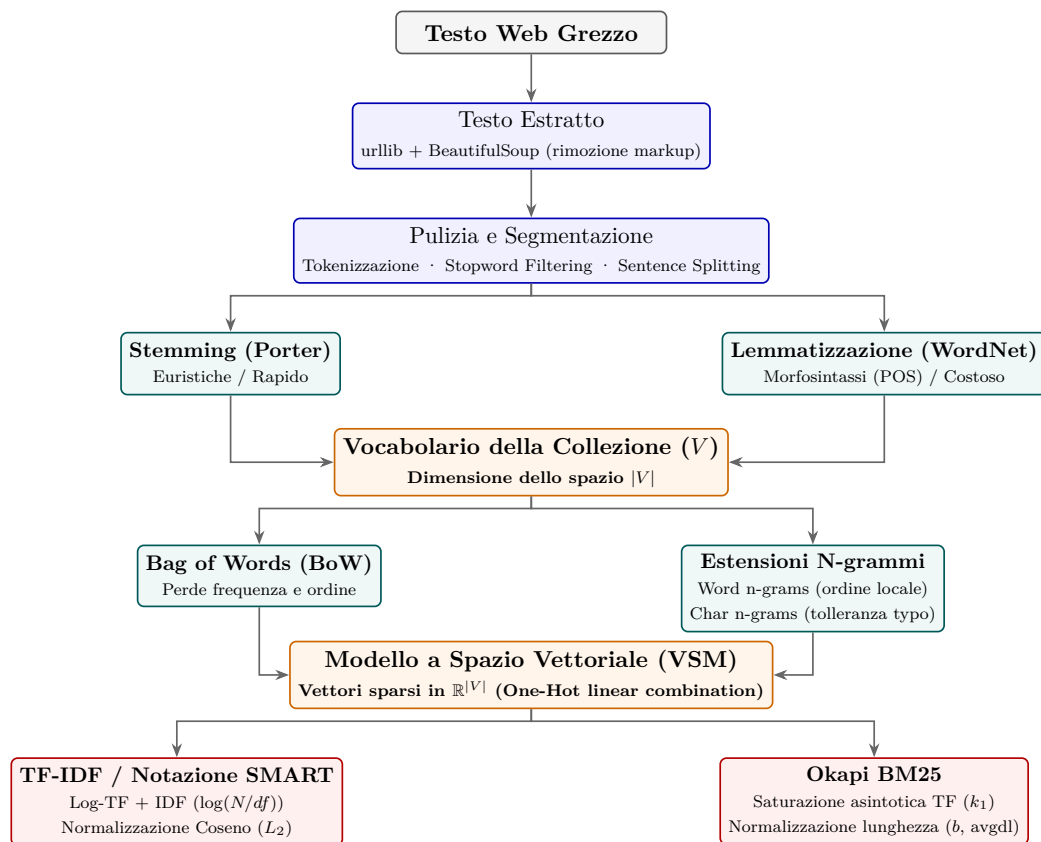


Figura 4.2: Mappa concettuale integrata della pipeline di Information Retrieval: dal crawling testuale ai modelli di scoring vettoriali e probabilistici.

Capitolo 5

Architettura dei Motori di Ricerca: Indicizzazione e Query Processing

Nei capitoli precedenti sono stati formalizzati i principi cardine dell'Information Retrieval: dalla valutazione dell'efficacia delle graduatorie (Capitolo 1) alle proprietà statistiche del linguaggio (Capitolo 3), fino alla definizione formale delle funzioni di rilevanza nello Spazio Vettoriale e del modello probabilistico Okapi BM25 (Capitolo 4). Tali modelli definiscono *cosa* calcolare per stimare la pertinenza di un documento rispetto a una query.

In questo capitolo si affronta la sfida ingegneristica complementare: **come calcolare tale graduatoria in tempo reale su architetture hardware moderne**, garantendo tempi di risposta nell'ordine di poche decine di millisecondi su archivi composti da miliardi di documenti. Vengono analizzati il paradigma di retrieval a due stadi, la risoluzione dell'intrinseca sparsità della matrice termini-documenti attraverso il confronto tra *Forward Index* e *Indice Invertito*, le tecniche di disaccoppiamento e compressione fisica degli stream posizionali, le patologie di *Query Expansion* e, infine, il confronto analitico tra gli algoritmi di scansione *Term-at-a-Time* (TAAT) e *Document-at-a-Time* (DAAT).

5.1 Architettura di Sistema e Paradigma a Due Stadi

Un motore di ricerca su scala Web deve conciliare requisiti architetturali apparentemente inconciliabili: processare collezioni testuali non strutturate di dimensioni sterminate garantendo al contempo una latenza di risposta

interattiva ($< 50\text{--}100$ ms) per ciascuna interrogazione utente. Per soddisfare tali vincoli, il flusso computazionale viene rigidamente ripartito tra processi asincroni massivi (**fase offline**) e processi sincroni a latenza vincolata (**fase online**), come schematizzato nella Sezione 1.1.

5.1.1 Pipeline Offline: Ingestione, Indicizzazione e Compressione

La pipeline offline opera sulla collezione documentale $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$ attraverso tre macro-fasi:

1. **Text Processing e Parsing:** I flussi testuali grezzi raccolti tramite web crawling vengono ripuliti dal markup, tokenizzati e normalizzati secondo le pipeline analizzate nella Sezione 3.4.
2. **Generazione del Vocabolario e dell'Indice:** Viene estratto il lessico $\mathcal{V} = \{t_1, t_2, \dots, t_M\}$ e costruita la struttura associativa che mappa ciascun termine alle relative occorrenze nei documenti.
3. **Compressione Numerica Lossless:** Poiché l'overhead dimensionale dell'indice grezzo può eguagliare o superare la dimensione del corpus di partenza, vengono applicati algoritmi di compressione per interi volti a massimizzare la residenza dei dati nei livelli alti della gerarchia di memoria (RAM e cache di sistema, come approfondito nel Capitolo 2).

5.1.2 Pipeline Online e Paradigma di Retrieval a Due Stadi

L'architettura moderna di risoluzione delle query adotta una pipeline a due stadi (*Two-Stage Retrieval Pipeline*), volta a bilanciare l'efficienza computazionale del recupero lessicale con l'elevata accuratezza semantica dei modelli predittivi complessi:

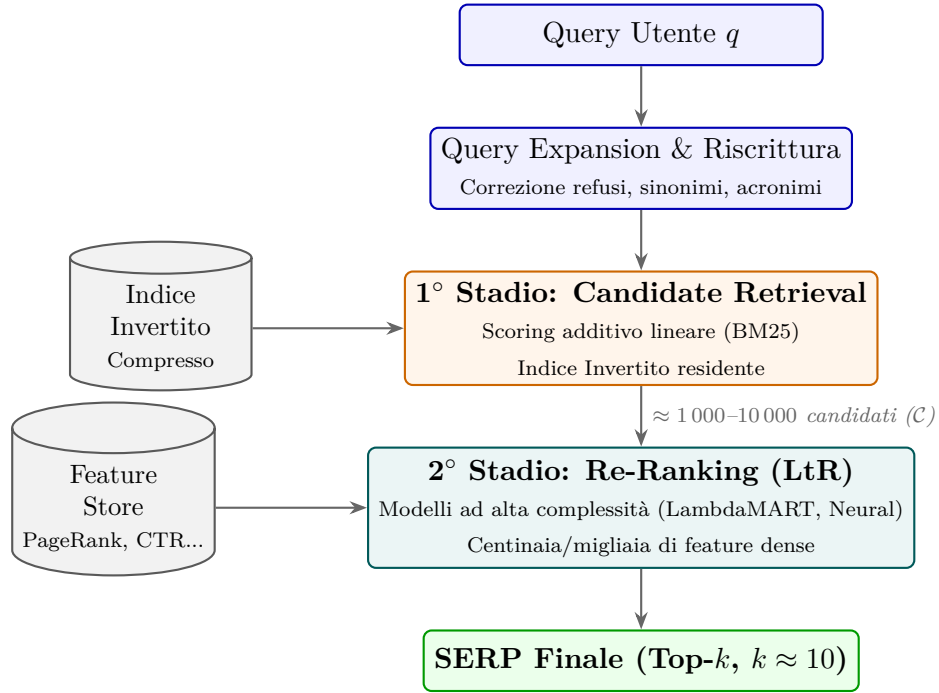


Figura 5.1: Architettura del paradigma di retrieval a due stadi: filtraggio massivo ad alta efficienza (primo stadio) e riordinamento semantico ad alta accuratezza (secondo stadio).

1. **Primo Stadio (Candidate Retrieval):** opera sull'intera collezione ($N \sim 10^9\text{--}10^{10}$ documenti). Impiega funzioni di scoring additive veloci (tipicamente Okapi BM25) per estrarre rapidamente un sottoinsieme ristretto di candidati promettenti:

$$\mathcal{C} \subset \mathcal{D}, \quad |\mathcal{C}| \approx 1\,000\text{--}10\,000 \text{ documenti.} \quad (5.1)$$

2. **Secondo Stadio (Re-Ranking / Heavy Scoring):** opera esclusivamente sull'insieme $\mathcal{C}$. Applica modelli di Machine Learning complessi (*Learning-to-Rank*, quali LambdaMART [4] o reti neurali basate su Transformer) valutando feature contestuali, grafiche, storiche e di autorità documentale per produrre la graduatoria finale Top- k ($k \approx 10$).

5.2 Modelli di Scoring: Prodotto Scalare e Connessione con il Dense Retrieval

5.2.1 Richiamo dei Limiti del Modello Booleano e di Jaccard

Nei sistemi delle prime generazioni, il reperimento dei documenti poggiava sul modello booleano esatto. Come formalizzato nella Sezione 4.1, tale approccio soffre della grave anomalia del *Feast or Famine*: una congiunzione stringente (AND) svuota l'insieme soluzione, mentre una disgiunzione (OR) restituisce centinaia di migliaia di risultati non graduati. Analogamente, l'applicazione ingenua del coefficiente insiemistico di Jaccard (analizzato in Sezione 4.2) fallisce poiché penalizza arbitrariamente i documenti lunghi e non discrimina tra parole comuni e parole altamente informative.

5.2.2 Il Prodotto Scalare come Misura Unificante di Rilevanza

Per superare tali limitazioni, lo Spazio Vettoriale (VSM) e i moderni schemi di ponderazione (formalizzati nel Capitolo 4) proiettano documenti e query in vettori pesati, trasformando la misura di pertinenza nel calcolo di un **prodotto scalare**:

$$\text{Score}(q, d) = \mathbf{q} \cdot \mathbf{d} = \sum_{t \in \mathcal{V}} w(t, q) \cdot w(t, d). \quad (5.2)$$

Poiché per una query standard i termini non menzionati hanno peso $w(t, q) = 0$, la sommatoria (5.2) si restringe ai soli termini condivisi:

$$\text{Score}(q, d) = \sum_{t \in q \cap d} w(t, q) \cdot w(t, d). \quad (5.3)$$

Questa formulazione evidenzia una profonda convergenza concettuale tra l'Information Retrieval lessicale classico e il moderno retrieval neurale:

- **Lexical Sparse Retrieval:** I vettori appartengono allo spazio $\mathbb{R}^{|\mathcal{V}|}$, dove ciascuna dimensione corrisponde a un lessema univoco del vocabolario ($|\mathcal{V}| \sim 10^7$). I vettori sono fortemente sparsi e lo score è calcolato tramite pesi statistici (TF-IDF, BM25).
- **Dense Neural Retrieval:** Modelli basati su Transformer (es. bi-encoder, Sentence-BERT) mappano testi e query in vettori continui densi di dimensionalità ridotta $\mathbf{v}_q, \mathbf{v}_d \in \mathbb{R}^k$ ($k \approx 768$ o $1\,536$). La

rilevanza semantica è determinata dal prodotto scalare o similarità coseno:

$$\text{Score}_{\text{neural}}(q, d) = \mathbf{v}_q \cdot \mathbf{v}_d = \sum_{i=1}^k v_{q,i} \cdot v_{d,i}. \quad (5.4)$$

In entrambi i paradigmi, l'operazione fondamentale a runtime è l'aggregazione di prodotti scalari su collezioni su larga scala.

5.3 Strutture Dati ad Alte Prestazioni: Forward Index vs. Indice Invertito

5.3.1 La Matrice Termini–Documenti e il Problema della Sparsità

Consideriamo la matrice globale termini–documenti $M \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{D}|}$. I parametri dimensionali di un motore di ricerca web reale evidenziano l'impossibilità fisica di allocare e scansionare tale struttura in forma densa:

- Dimensione del lessico: $|\mathcal{V}| \sim 10^7$ termini unici.
- Dimensione della collezione: $|\mathcal{D}| \sim 10^9\text{--}10^{10}$ documenti web.
- Numero totale di celle: $|\mathcal{V}| \times |\mathcal{D}| \approx 10^{16}\text{--}10^{17}$ posizioni.

Poiché un documento medio contiene tra 10^2 e 10^3 vocaboli distinti, la densità di elementi non nulli è infinitesima:

$$\frac{\text{elementi non nulli}}{|\mathcal{V}| \times |\mathcal{D}|} \ll 0.001\%. \quad (5.5)$$

Oltre il 99.99% delle celle contiene il valore zero. Memorizzare la matrice in formato denso comporterebbe la saturazione immediata della memoria secondaria e una complessità temporale di valutazione query proibitiva, pari a $\mathcal{O}(|\mathcal{V}| \cdot |\mathcal{D}|)$, costringendo la CPU a esaminare miliardi di documenti irrilevanti.

5.3.2 Rappresentazione tramite Vettori Sparsi: Forward Index vs. Indice Invertito

Per eliminare gli zeri, la matrice viene memorizzata come un insieme di vettori sparsi composti da coppie (Identificatore, Valore). L'orientamento di serializzazione determina due strutture complementari, sintetizzate nella Tabella 5.1:

- **Forward Index (Column-Wise / Document-Oriented):** memorizza per ciascun documento la lista dei termini che vi compaiono, ad esempio $d_1 \rightarrow [(t_2, w_2), (t_7, w_7)]$.
- **Indice Invertito (Row-Wise / Term-Oriented):** memorizza per ciascun termine la lista dei documenti in cui ricorre, ad esempio $t_1 \rightarrow [(d_2, w_1), (d_5, w_5)]$.

Proprietà	Forward Index	Indice Invertito
Orientamento	Per colonna (Document-Oriented).	Per riga (Term-Oriented).
Struttura logica	$\text{DocID} \rightarrow [(t_a, \text{tf}_a), \dots]$	$t \rightarrow [(\text{DocID}_i, \text{tf}_i), \dots]$
Complessità query	$\mathcal{O}(\mathcal{D})$: richiede ispezione di tutti i documenti.	$\mathcal{O}(\sum_{t \in q} \mathcal{L}(t))$: accede solo ai documenti candidati.
Ruolo nel motore	Memorizzazione metadati, feature store, snippet generation.	Cuore computazionale del First-Stage Retrieval (matching rapido).

Tabella 5.1: Confronto sistematico tra Forward Index e Indice Invertito.

5.4 Anatomia, Layout di Memoria e Compressione dell’Indice Invertito

Un indice invertito di livello industriale si articola in due componenti fisiche distinte: il **Dizionario** (o Lessico) e l’insieme delle **Posting List**.

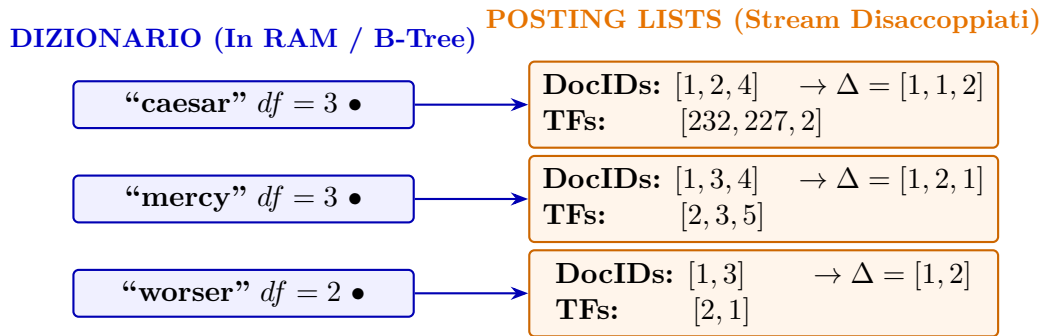


Figura 5.2: Anatomia di un indice invertito con disaccoppiamento fisico tra stream dei DocID (compressi tramite d -gaps) e stream delle frequenze tf.

5.4.1 Il Dizionario (Lexicon)

Il dizionario memorizza il vocabolario $\mathcal{V}$. Per ciascun termine t , registra la stringa lessicale, la Document Frequency df_t (utilizzata per ricavare l'IDF precalcolato) e il puntatore di memoria o l'offset su storage secondario che localizza la corrispondente posting list. Dato che $|\mathcal{V}|$ è dell'ordine di milioni di lessemi, il dizionario viene mantenuto interamente residente in **RAM**, indicizzato tramite strutture di accesso rapido come B-Tree, Trie compressi o tabelle hash.

5.4.2 Le Posting List: Disaccoppiamento Fisico degli Stream

Concettualmente, la posting list associata al termine t è una sequenza di tuple:

$$\mathcal{L}(t) = [\langle d_1, \text{tf}_{t,d_1} \rangle, \langle d_2, \text{tf}_{t,d_2} \rangle, \dots]. \quad (5.6)$$

A livello implementativo, memorizzare array continui di strutture composte (es. struct C++ contenenti `uint32_t doc_id` e `uint32_t tf`) è altamente sub-ottimale. Per massimizzare i fattori di compressione numerica, i motori di ricerca operano il **disaccoppiamento fisico in stream paralleli**:

1. Stream dei DocID (Monotono Crescente):

$$\mathbf{D}_t = [\text{DocID}_1, \text{DocID}_2, \dots, \text{DocID}_m] \quad \text{con } \text{DocID}_i < \text{DocID}_{i+1}. \quad (5.7)$$

Sfruttando la stretta monotonicità, i valori assoluti vengono convertiti nelle differenze tra identificatori contigui, dette **d -gaps** o delta:

$$\Delta_1 = \text{DocID}_1, \quad \Delta_i = \text{DocID}_i - \text{DocID}_{i-1} \quad (\forall i \geq 2). \quad (5.8)$$

I delta generati sono interi piccoli (spesso $\Delta \leq 5$ per termini ad alta frequenza), comprimibili ad altissima densità tramite algoritmi dedicati (*Variable Byte*, *Elias-Fano*, *PForDelta*, *Binary Interpolative Coding* [3]).

2. Stream delle Frequenze (Non Ordinato):

$$\mathbf{F}_t = [\text{tf}(t, d_1), \text{tf}(t, d_2), \dots, \text{tf}(t, d_m)]. \quad (5.9)$$

Le frequenze locali non godono di ordinamento; tuttavia, per la legge di Zipf, la loro distribuzione empirica è fortemente concentrata su valori minimi (1, 2, 3). Esse vengono compresse simmetricamente tramite codici a lunghezza variabile senza richiedere la differenziazione delta.

5.4.3 Trade-Off Architetture: Interi Compresi vs. Float Precalcolati

Una decisione progettuale critica consiste nello stabilire se memorizzare nell'indice il punteggio finale precalcolato $w(t, d) \in \mathbb{R}$ a virgola mobile oppure l'intero grezzo $\text{tf}(t, d) \in \mathbb{N}$:

Parametro	Punteggio Float Precalcolato	Intero Grezzo TF Compresso
Occupazione Spazio	32 o 64 bit per posting (float/double).	Intero compresso (spesso < 8 bit medi per posting).
Comprimibilità	Scarsa o nulla: mantissa float a entropia quasi-casuale.	Elevatissima: concentrazione statistica su delta piccoli.
Flessibilità Ranking	Nulla: vincola l'indice alla specifica formula usata a build-time.	Totale: consente di calcolare qualsiasi funzione (BM25, varianti) a runtime.
Throughput CPU	Nessun calcolo a runtime (lettura diretta).	Pochi cicli ALU per posting per calcolare la formula.

Tabella 5.2: Confronto di sistema tra memorizzazione float e interi compresi.

Risoluzione del Vincolo Memory-Bound

Come analizzato nel Capitolo 2, i sistemi di Information Retrieval su larga scala sono rigorosamente **memory-bound**: il vero fattore limitante delle prestazioni è la banda di trasferimento dalla memoria principale (DRAM) ai registri della CPU (≈ 80 ns di latenza), non il throughput dell'unità aritmetica (ALU, 0.5 ns). Ridurre la dimensione dei posting attraverso la compressione degli interi abbassa il volume di byte trasferiti sul bus di memoria; il modesto costo computazionale richiesto per valutare la formula a runtime (es. BM25) viene ampiamente compensato dal risparmio di banda.

5.5 Tipologie di Query, Indici Posizionali e Riscrittura del Testo

5.5.1 Phrase Queries e la Necessità dell'Indice Posizionale

Nelle ricerche per frase esatta (*Phrase Queries*), la query è racchiusa tra virgolette (es. "neural network architecture"). L'utente impone che i termini compaiano nel medesimo documento, nello stesso ordine e in posizioni adiacenti:

$$\text{pos}(t_{i+1}) = \text{pos}(t_i) + 1. \quad (5.10)$$

L'indice invertito standard a sole frequenze (DocID, tf) non permette di validare questo vincolo, avendo scartato la posizione delle parole. Per supportare le phrase queries, la struttura viene estesa a **Indice Posizionale**, memorizzando per ogni posting la lista degli offset ordinali:

$$t \longrightarrow [\langle \text{DocID}_k, \text{tf}_k, [\text{pos}_1, \text{pos}_2, \dots, \text{pos}_{\text{tf}}] \rangle]. \quad (5.11)$$

Esempio 5.5.1 (Intersezione Posizionale per Phrase Match). Si consideri la query "neural network":

- Posting per "neural": $\dots \rightarrow \langle \text{Doc } 14, \text{tf} = 2, \text{pos} = [4, 21] \rangle \rightarrow \dots$
- Posting per "network": $\dots \rightarrow \langle \text{Doc } 14, \text{tf} = 3, \text{pos} = [5, 89, 102] \rangle \rightarrow \dots$

L'algoritmo esegue l'intersezione su DocID = 14 e confronta le liste posizionali: poiché $\text{pos}(\text{neural}) = 4$ e $\text{pos}(\text{network}) = 5$, la condizione $\text{pos}(t_{i+1}) = \text{pos}(t_i) + 1$ è soddisfatta e il documento viene validato.

Overhead spaziale: L'inclusione delle posizioni incrementa l'impronta complessiva su disco dell'indice invertito di un fattore compreso tra $2\times$ e $3\times$.

5.5.2 Moduli di Query Expansion e il Caso Studio "Babbo Francesco"

I motori di ricerca industriali intercettano la query grezza dell'utente attraverso un modulo di **Query Expansion** e riscrittura, con l'obiettivo di correggere refusi, risolvere acronimi ed espandere sinonimi contestuali.

Tuttavia, espansioni lessicali non supervisionate basate su matrici di co-occorrenza o regole euristiche possono innescare gravi derive semantiche (*semantic drift*), come illustrato dal celebre caso riscontrato nei motori commerciali in lingua italiana:

1. **Query Utente:** L'utente inserisce la query a intento personale/familiare "babbo francesco".
2. **Espansione Sinonimica:** Il modulo associa il termine dialettale/regionale *babbo* al sinonimo standard *papà*.
3. **Normalizzazione e Troncamento Accenti:** La pipeline rimuove gli accenti trasformando *papà* in *papa* (omografo non accentato che designa il Pontefice).
4. **Query Riscritta all'Indice:** La disgiunzione risultante inviata al motore diventa ("babbo" OR "papà" OR "papa") AND "francesco".
5. **Scoring & Authority Ranking:** L'entità globale "Papa Francesco" possiede un'autorevolezza a priori (PageRank) e una co-occorrenza web sterminata rispetto a qualunque testo privato.
6. **Risultato Top-1 Restituito:** La SERP collassa restituendo in prima posizione la voce enciclopedica su *Papa Francesco (Jorge Mario Bergoglio)*, stravolgendo completamente l'intento informativo dell'utente.

5.5.3 Stemming vs. Query Expansion

È fondamentale chiarire la linea di demarcazione tra normalizzazione morfologica ed espansione semantica:

- **Stemming e Lemmatizzazione:** Operano riduzioni sintattiche all'interno della medesima radice etimologica (es. *running*, *runs* → *run*, come descritto in Sezione 3.6). Non possono collegare vocaboli con radici distinte aventi identico significato (es. *babbo* e *papà*).
- **Approccio Moderno:** I sistemi scalabili contemporanei riducono l'uso di stemming aggressivo in fase di indicizzazione (per preservare la specificità semantica dei termini nell'indice) e gestiscono le varianti flessive e sinonimiche a runtime tramite modelli linguistici neurali di espansione contestuale.

5.6 Algoritmi di Query Processing: Term-at-a-Time (TAAT) vs. Document-at-a-Time (DAAT)

Individuate le posting list associate ai termini di query $q = \{t_1, t_2, \dots, t_m\}$, il motore deve scansionare i dati ed estrarre i primi k documenti con punteggio

più elevato per popolare una coda con priorità (min-heap di dimensione fissa k). Esistono due paradigmi algoritmici fondamentali: **Term-at-a-Time (TAAT)** e **Document-at-a-Time (DAAT)**.

Term-at-a-Time (TAAT) — Scansione Orizzontale $\mathcal{L}(t_1): [(1, 2) \rightarrow (4, 1) \rightarrow (9, 3)] \Rightarrow$ Aggiorna accumulatori $\mathcal{L}(t_2): [(2, 1) \rightarrow (4, 5) \rightarrow (7, 2)] \Rightarrow$ Somma su accumulatori $\mathcal{L}(t_3): [(1, 4) \rightarrow (3, 2) \rightarrow (9, 1)] \Rightarrow$ Somma su accumulatori Fine: Estrazione Top- k dalla tabella degli accumulatori.	Document-at-a-Time (DAAT) — Scansione Verticale Cursori: $p_1 \rightarrow \text{Doc 1}, p_2 \rightarrow \text{Doc 2}, p_3 \rightarrow \text{Doc 1}$ Step 1: $d_{\min} = 1$. Calcola Score finale per Doc 1. Inserisce direttamente in Min-Heap Top- k . Avanza puntatori p_1 e p_3 . Step 2: $d_{\min} = 2$. Calcola Score finale per Doc 2...
--	--

Figura 5.3: Confronto visivo delle modalità di scorrimento delle posting list: TAAT orizzontale sequenziale vs. DAAT verticale sincrono.

5.6.1 La Strategia Term-at-a-Time (TAAT)

Nell'approccio TAAT, le posting list vengono elaborate in modo **sequenziale, una alla volta**:

1. Viene allocata una struttura di **accumulatori** $A[\text{DocID}] \in \mathbb{R}$ (tabella hash o array denso indicizzato per DocID).
2. Il motore apre la posting list del primo termine $\mathcal{L}(t_1)$ e itera su tutti i suoi posting, calcolando il punteggio parziale e incrementando l'accumulatore corrispondente.
3. Conclusa $\mathcal{L}(t_1)$, si passa a $\mathcal{L}(t_2)$ sommando i nuovi contributi, procedendo fino all'ultima lista $\mathcal{L}(t_m)$.
4. Al termine, si estraggono i top- k documenti scansionando la struttura degli accumulatori tramite un min-heap.

Algorithm 2: Term-at-a-Time (TAAT)

Dati: Query $q = \{t_1, t_2, \dots, t_m\}$, Indice Invertito, intero k
Risultato: Min-Heap H contenente i Top- k documenti per punteggio

```
1  $A \leftarrow$  InizializzaTabellaAccumulatori() // Mappa DocID  $\rightarrow$  Score Parziale
2 foreach termine  $t_i \in q$  do
3    $\mathcal{L}_i \leftarrow$  RecuperaPostingList( $t_i$ )
4    $\text{idf}_i \leftarrow$  CalcolaIDF( $t_i$ )
5   foreach  $\langle d, \text{tf} \rangle \in \mathcal{L}_i$  do
6      $s_{\text{parz}} \leftarrow$  CalcolaContributo( $\text{tf}, \text{idf}_i$ )
7     if  $d \in A$  then
8        $A[d] \leftarrow A[d] + s_{\text{parz}}$ 
9     else
10       $A[d] \leftarrow s_{\text{parz}}$ 
11  $H \leftarrow$  InizializzaMinHeap(capacità =  $k$ )
12 foreach coppia  $(d, \text{score}) \in A$  do
13   if  $H.\text{Dimensione}() < k$  then
14      $H.\text{Inserisci}(d, \text{score})$ 
15   else
16     if  $\text{score} > H.\text{MinimoPunteggio}()$  then
17        $H.\text{EstraiMinimo}()$ 
18        $H.\text{Inserisci}(d, \text{score})$ 
19 return  $H$ 
```

Analisi prestazionale di TAAT:

- **Vantaggi:** Accesso strettamente sequenziale alla memoria. Ciascuna posting list viene letta come un blocco contiguo, massimizzando l'efficacia del prefetching hardware della CPU e la località di cache (Capitolo 2).
- **Svantaggi:** Footprint di memoria volatile elevatissimo. La tabella degli accumulatori deve contenere l'unione dei documenti di tutte le posting list: con termini ad alta Document Frequency, A può coinvolgere decine di milioni di record. Inoltre, è impossibile effettuare il *dynamic early pruning*, poiché uno score parziale basso potrebbe essere incrementato arbitrariamente dalle liste successive.

5.6.2 La Strategia Document-at-a-Time (DAAT)

Nell'approccio DAAT, le posting list di tutti i termini di query vengono elaborate **simultaneamente e in parallelo**, operando come un merge multi-cursore:

1. Viene posizionato un puntatore p_i all'inizio di ciascuna lista $\mathcal{L}(t_i)$.
2. A ogni iterazione, il sistema determina il minimo identificatore corrente:

$$d_{\min} = \min_{i \in \{1, \dots, m\}} \{p_i.\text{CurrentDocID}()\}. \quad (5.12)$$

3. Vengono identificati tutti i cursori posizionati su $d_{\min}$, calcolando immediatamente il **punteggio finale consolidato** del documento:

$$S(q, d_{\min}) = \sum_{i: p_i.\text{DocID}=d_{\min}} \text{ScoreTerm}(p_i.\text{TF}, t_i). \quad (5.13)$$

4. Il punteggio definitivo viene confrontato all'istante con il Min-Heap di taglia k : se $S(q, d_{\min}) > H.\text{Minimo}()$, il documento entra nella Top- k scalzando il vecchio minimo.
5. Tutti i cursori attestati su $d_{\min}$ avanzano al prossimo elemento.

Algorithm 3: Document-at-a-Time (DAAT)

Dati: Query $q = \{t_1, t_2, \dots, t_m\}$, Indice Invertito, intero k
Risultato: Min-Heap H contenente i Top- k documenti per punteggio

```
1 cursori  $\leftarrow$  InizializzaCursoriPerOgniLista( $q$ )
2  $H \leftarrow$  InizializzaMinHeap(capacità =  $k$ )
3 while EsistonoCursoriAttivi(cursori) do
4      $d_{\min} \leftarrow \min\{p.\text{CurrentDocID}() \mid p \in \text{cursori} \wedge p.\text{ÈAttivo}()\}$ 
5      $\text{score}_{\text{tot}} \leftarrow 0.0$ 
6     foreach  $p \in \text{cursori}$  do
7         if  $p.\text{ÈAttivo}() \wedge p.\text{CurrentDocID}() == d_{\min}$  then
8              $\text{score}_{\text{tot}} \leftarrow$ 
9                  $\text{score}_{\text{tot}} + \text{CalcolaContributo}(p.\text{CurrentTF}(), p.\text{Termine}())$ 
10                 $p.\text{Avanza}()$ 
11         // Il documento è completo: inserimento immediato
12         nell'heap
13     if  $H.\text{Dimensione}() < k$  then
14          $H.\text{Inserisci}(d_{\min}, \text{score}_{\text{tot}})$ 
15     else
16         if  $\text{score}_{\text{tot}} > H.\text{MinimoPunteggio}()$  then
17              $H.\text{EstraiMinimo}()$ 
18              $H.\text{Inserisci}(d_{\min}, \text{score}_{\text{tot}})$ 
19 return  $H$ 
```

Analisi prestazionale di DAAT:

- **Vantaggi:** Memoria di lavoro trascurabile ($\mathcal{O}(k)$ per il Min-Heap e $\mathcal{O}(m)$ per lo stato dei cursori), senza alcuna tabella intermedia di accumulo. Supporto nativo all'**Early Pruning dinamico**: conoscendo la soglia minima corrente dell'heap $\theta = H.\text{MinimoPunteggio}()$, algoritmi avanzati come **WAND** (*Weak AND*) o **Block-Max WAND** possono stimare i limiti superiori (*upper bounds*) di punteggio residuo e saltare a blocchi intere sequenze di posting che matematicamente non possono superare θ , evitando decompressioni superflue.
- **Svantaggi:** Accesso alla memoria frammentato su più stream contigui, con frequenti salti tra puntatori che riducono l'efficacia del caching rispetto alla lettura lineare di TAAT.

5.6.3 Quadro Sinottico Comparativo

Dimensione	Term-at-a-Time (TAAT)	Document-at-a-Time (DAAT)
Direzione di Scansione	Orizzontale (lista per lista, sequenziale).	Verticale (documento per documento, parallelo).
Stato del Punteggio	Punteggi parziali progressivi in memoria.	Punteggio completo definitivo per ciascun d .
Occupazione di RAM	Elevata ($\mathcal{O}(\bigcup \mathcal{L}(t_i))$ accumulatori).	Minima ($\mathcal{O}(k)$ per il Min-Heap Top- k).
Località di Cache	Ottima (scansione continua di stream compresso).	Frammentata (salti alternati tra m cursori).
Early Termination	Complessa e poco efficace.	Naturale ed efficientissima (WAND, Block-Max).
Scelta Industriale Tipica	Retrieval su GPU / architetture parallele massive.	Core retrieval standard su CPU (Lucene, Elasticsearch).

Tabella 5.3: Sintesi comparativa delle proprietà architettureali di TAAT e DAAT.

5.6.4 Rilevanza nei Motori Vettoriali e Sistemi Moderni (Vector Search)

Il dualismo tra TAAT e DAAT non appartiene unicamente alla tradizione dell'IR classico, ma definisce le architetture dei motori di ricerca neurale contemporanei:

- **Sparse Neural Retrieval (es. SPLADE [6]):** Quando modelli di deep learning predicono distribuzioni di pesi sparsi su vocabolari estesi, i motori riadattano indici invertiti processati con algoritmi DAAT potenziati da WAND.
- **Acceleratori Tensoriali e GPU:** In ambienti basati su GPU, il paradigma TAAT torna competitivo per via dell'altissimo throughput garantito da operazioni vettoriali contigue su blocchi di memoria regolari.

- **Vector Search Engines Dedicati (es. Qdrant):** Motori di ricerca vettoriale moderni integrano indici invertiti di supporto per combinare filtri booleani e metadati con indici di prossimità geometrica (k -ANN su grafi HNSW), riutilizzando i pattern di scansione DAAT/TAAT per garantire un'efficienza congiunta nel primo stadio di recupero.

Bibliografia

- [1] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [2] Stefan Büttcher, Charles L. A. Clarke, and Gordon V. Cormack. *Information Retrieval: Implementing and Evaluating Search Engines*. MIT Press, 2010.
- [3] Alistair Moffat and Luke Stuiver. Binary interpolative coding for effective index compression. *Information Processing & Management*, 36(5):667–687, 2000.
- [4] Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.
- [5] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020.
- [6] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for information retrieval. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2288–2292, 2021.
- [7] Cyril W. Cleverdon. The Cranfield tests on index language devices. *Aslib Proceedings*, 19(6):173–194, 1967.
- [8] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of retrieval techniques. *ACM Transactions on Information Systems (TOIS)*, 20(4):422–446, 2002.